Atelier 4

Agrégation et index avec MongoDB

Cet atelier a trois objectifs : calculer des statistiques sur des commandes avec la **chaîne d'agrégation** de MongoDB ; accélérer les recherches avec des **index** ; vérifier l'effet d'un index en lisant le **plan d'exécution** d'une requête, c'est-à-dire la liste des opérations que le serveur a réellement faites pour y répondre. Les parties 1 et 2 se font dans `mongosh`, l'interpréteur de commandes de MongoDB. La partie 3 refait une partie du travail dans Compass, l'interface graphique de MongoDB. La partie 4 fait le même travail depuis un programme Python, comme le ferait une application : lancer une chaîne d'agrégation, ranger son résultat, créer des index, mesurer une requête et écrire en masse. La partie 5, pour aller plus loin, reprend avec des explications courtes ce que le chapitre 3 du cours (chapitre 4 dans la version complète) ne fait que résumer : comment un index est rangé et ce qu'il coûte, la lecture détaillée d'un plan, les index texte, géographiques, hachés et génériques, et le choix d'un index. Un mémo des commandes et des fonctions termine l'atelier.

Ce qu'il faut avant de commencer

— Un serveur MongoDB 8.0 qui tourne sur votre machine et écoute sur le port 27017 : soit installé directement, soit dans un conteneur Docker nommé `mongo`, créé par la commande ci-dessous (s'il est arrêté, `docker start mongo` le relance).

```
docker run -d --name mongo -p 127.0.0.1:27017:27017 -v mongo-data:/data/db mongo:8.0
```

— L'interpréteur `mongosh`, installé sur votre machine ou pris dans le conteneur.

— MongoDB Compass, pour la partie 3.

— Python et le pilote PyMongo 4.18.2, pour la partie 4, installés à l'atelier 2 : dans l'environnement virtuel `~/env-mongo`, ou dans le conteneur `python-mongo`.

Fichiers fournis

— `donnees-boutique.js` crée la base `boutique` : 200 produits et 1 000 commandes, toujours les mêmes. Relancer ce script remet la base dans son état de départ : il la supprime entièrement, `commandes_volume` comprise, qu'il faut alors recharger avec `donnees-volume.js`.

— `donnees-volume.js` ajoute à cette base la collection `commandes_volume` : 200 000 commandes de structure voisine (voir l'exercice 8), assez pour que l'effet d'un index se mesure.

— `atelier04-mesure.py`, `atelier04-masse.py` et `atelier04-rapport.py` (dossier `programmes`) : les trois programmes Python complets de la partie 4.

Conventions

— Sauf mention contraire, chaque commande se tape dans `mongosh`, à l'invite `boutique>`.

— Faites les exercices **dans l'ordre** : certains créent des index ou des champs dont les suivants tiennent compte.

— Les durées (`ms`) dépendent de votre machine ; les nombres de documents et de clés lus, eux, doivent être identiques aux valeurs de ce document.

— `mongosh` peut répartir un même résultat sur plusieurs lignes, et les documents d'un `$group` non trié peuvent arriver dans un autre ordre.

Partie 1 : la chaîne d'agrégation

Chaîne d'agrégation (*aggregation pipeline*)

Une chaîne d'agrégation est une suite d'**étapes** (*stages*). La première étape reçoit tous les documents de la collection ; chaque étape transforme les documents qu'elle reçoit (elle en garde une partie, les regroupe, les trie, leur ajoute des champs) et passe son résultat à l'étape suivante ; `mongosh` affiche le résultat de la dernière. On l'écrit `db.commandes.aggregate([ étape1, étape2, … ])` : un tableau qui contient un document par étape. Dans une étape, une chaîne qui commence par un dollar, comme `"$client.ville"`, désigne la valeur de ce champ dans le document en cours.

Les étapes utilisées dans cet atelier, avec la clause SQL qui joue le même rôle :

Étape	Ce qu'elle fait	En SQL
`$match`	garde les documents qui vérifient une condition	`WHERE`
`$group`	regroupe les documents qui ont la même valeur pour la clé de regroupement, écrite dans `_id`, et calcule des valeurs pour chaque groupe	`GROUP BY`
`$sort`	trie les documents (1 : ordre croissant, -1 : décroissant)	`ORDER BY`
`$limit`	garde les *n* premiers documents	`LIMIT`
`$count`	remplace tous les documents par un seul, qui contient leur nombre	`COUNT(*)`
`$project`, `$set`	choisit les champs à garder (`$project`), ajoute un champ calculé (`$set`)	`SELECT`
`$unwind`	fait une copie du document pour chaque élément d'un tableau	(aucune)
`$lookup`	va chercher des documents dans une autre collection	`LEFT JOIN`
`$merge`	écrit le résultat dans une collection au lieu de l'afficher, en remplaçant ou en complétant les documents déjà présents	`MERGE`

Exercice 1 : Préparer les données

1. Ouvrez un terminal de votre système (PowerShell sous Windows, Terminal sous macOS ou Linux) dans le dossier qui contient les deux fichiers fournis. Si MongoDB est installé directement sur la machine, chargez les deux fichiers :

```
mongosh --file donnees-boutique.js
mongosh --file donnees-volume.js
```

Si MongoDB tourne dans le conteneur `mongo`, copiez d'abord les fichiers dans le conteneur, puis lancez-les avec le `mongosh` du conteneur :

```
docker cp donnees-boutique.js mongo:/tmp/
docker cp donnees-volume.js mongo:/tmp/
docker exec -it mongo mongosh --file /tmp/donnees-boutique.js
docker exec -it mongo mongosh --file /tmp/donnees-volume.js
```

Le second script dure de quelques secondes à une minute selon la machine : il insère les 200 000 commandes par lots de 10 000.

Vous devez obtenir

Chaque script affiche une ligne quand il a fini :

```
Base boutique chargée : 200 produits, 1000 commandes.
Collection commandes_volume chargée : 200000 commandes.
```

2. Ouvrez `mongosh` dans le même terminal : tapez `mongosh`, ou `docker exec -it mongo mongosh` avec Docker. L'invite `test>` indique que vous êtes sur la base `test`, choisie par défaut. Passez sur la base `boutique` et vérifiez son contenu :

```
use boutique
show collections
db.commandes.getIndexes()
```

Vous devez obtenir

```
boutique> show collections
commandes
commandes_volume
produits
boutique> db.commandes.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

L'invite devient `boutique>`. La collection `commandes` n'a qu'un index, sur le champ `_id` : MongoDB le crée pour toute collection.

3. Affichez une commande, puis le premier produit qu'elle contient :

```
db.commandes.findOne({ _id: 10001 })
db.produits.findOne({ ref: "P-012" }, { _id: 0 })
```

Vous devez obtenir

```
{ _id: 10001,
  client: { id: 8, nom: 'Client 8', ville: 'Tanger' },
  date: ISODate('2026-01-02T00:00:00.000Z'),
  statut: 'payée',
  lignes: [
    { ref: 'P-012', libelle: 'Maison n°12', prix: Decimal128('454.00'), qte: 2 },
    { ref: 'P-029', libelle: 'Sport n°29', prix: Decimal128('83.00'), qte: 3 }
  ] }
{ ref: 'P-012', libelle: 'Maison n°12', categorie: 'Maison', prix: Decimal128('454.00'),
  stock: 6, tags: [ 'promo', 'nouveau' ], ajoute_le: ISODate('2025-01-13T00:00:00.000Z') }
```

Une commande contient son numéro (`_id`, de 10001 à 11000), le client, rangé dans la commande sous la forme d'un sous-document (numéro, nom et ville, parmi cinq villes), la date, le statut (`payée` ou `annulée`) et le tableau `lignes`, qui a une entrée par produit acheté : référence, libellé, prix unitaire et quantité. Le type `Decimal128` est un nombre décimal exact : il évite, sur des montants, les erreurs d'arrondi des

nombres à virgule flottante. Un produit porte en plus sa catégorie (cinq catégories de 40 produits), son stock (le nombre d'exemplaires en magasin) et un tableau de mots-clés `tags`. Le second argument `{ _id: 0 }` du dernier `findOne`, appelé **projection**, masque le champ `_id` du produit : c'est un identifiant `ObjectId`, différent sur chaque machine.

Exercice 2 : Filtrer, regrouper, trier

La notion : étapes et accumulateurs

Étape

Une étape est une opération de la chaîne : elle reçoit des documents, en produit d'autres, et ne voit que ce que l'étape précédente lui transmet. `$match` laisse passer certains documents sans les modifier ; `$group` remplace les documents qu'il reçoit par un document par groupe ; `$sort` renvoie les mêmes documents, dans un autre ordre.

Accumulateur

Dans `$group`, chaque champ autre que `_id` est calculé par un accumulateur. Celui-ci part d'une valeur de départ et la met à jour avec chaque document du groupe, dans l'ordre où les documents arrivent : `{ $sum: 1 }` part de 0 et ajoute 1 ; `{ $push: "$_id" }` part d'un tableau vide et y ajoute l'`_id` du document ; `{ $min: "$date" }` garde la plus petite date vue.

Exemple déroulé : six commandes

La chaîne suivante ne travaille que sur six commandes, pour que l'on puisse suivre chaque document d'une étape à l'autre. Tapez-la :

```
db.commandes.aggregate([
  { $match: { _id: { $in: [ 10001, 10003, 10006, 10008, 10010, 10011 ] }, statut: "payée" } },
  { $group: { _id: "$client.ville", commandes: { $sum: 1 }, ids: { $push: "$_id" },
              premiere: { $min: "$date" } } },
  { $sort: { commandes: -1 } }
])
```

`$in` retient les documents dont le champ vaut l'une des valeurs de la liste. Ce que fait chaque étape :

1. `$match` reçoit les 1 000 commandes et n'en laisse passer que cinq : les six de la liste, moins la commande 10010, annulée. Les cinq documents sortent inchangés :

_id	client.ville	statut	date
10001	Tanger	payée	02/01/2026
10003	Fès	payée	04/01/2026
10006	Tanger	payée	07/01/2026
10008	Fès	payée	09/01/2026
10010	Casablanca	annulée	11/01/2026 (écartée)
10011	Tanger	payée	12/01/2026

2. `$group` range chaque document dans le groupe de sa ville et met à jour les trois accumulateurs de ce groupe. Les documents arrivent dans l'ordre de leur `_id` :

Document reçu	Groupe	commandes	ids	premiere
10001	Tanger	1	[10001]	02/01/2026
10003	Fès	1	[10003]	04/01/2026
10006	Tanger	2	[10001, 10006]	02/01/2026
10008	Fès	2	[10003, 10008]	04/01/2026
10011	Tanger	3	[10001, 10006, 10011]	02/01/2026

Une fois tous les documents reçus, `$group` produit un document par groupe : deux documents, dont l'`_id` porte la ville.

3. `$sort` reçoit ces deux documents et met Tanger (3 commandes) avant Fès (2).

Vous devez obtenir

Pour voir la sortie d'une étape, lancez la chaîne arrêtée après elle. Arrêtée après `$group`, elle renvoie les deux groupes, dans un ordre qui peut varier ; complète, elle renvoie :

```
[ { _id: 'Tanger', commandes: 3, ids: [ 10001, 10006, 10011 ],
    premiere: ISODate('2026-01-02T00:00:00.000Z') },
  { _id: 'Fès', commandes: 2, ids: [ 10003, 10008 ],
    premiere: ISODate('2026-01-04T00:00:00.000Z') } ]
```

Exemple guidé : construire une chaîne étape par étape

On veut le nombre de commandes payées de chaque ville, de la ville qui en a le plus à celle qui en a le moins. On ajoute les étapes une à une, et on vérifie le résultat à chaque fois.

1. Écrivez la première étape seule. `$match` garde les commandes payées ; `$count`, ajouté pour l'instant, compte les documents qu'elle laisse passer et range ce nombre dans un champ dont on choisit le nom (ici `commandes`) :

```
db.commandes.aggregate([ { $match: { statut: "payée" } }, { $count: "commandes" } ])
```

Vous devez obtenir

```
[ { commandes: 900 } ]
```

900 commandes sur 1 000 sont payées ; les 100 autres sont annulées.

2. Remplacez `$count` par un regroupement par ville. Dans `$group`, le champ `_id` donne la clé de regroupement : ici la ville du client. Chaque autre champ est calculé par un **accumulateur** sur les documents du groupe : `{ $sum: 1 }` ajoute 1 pour chaque document, donc compte les commandes.

```
db.commandes.aggregate([
  { $match: { statut: "payée" } },
  { $group: { _id: "$client.ville", commandes: { $sum: 1 } } }
])
```

Vous devez obtenir

```
[ { _id: 'Marrakech', commandes: 200 }, { _id: 'Casablanca', commandes: 100 },
  { _id: 'Tanger', commandes: 200 },    { _id: 'Fès', commandes: 200 },
  { _id: 'Rabat', commandes: 200 } ]
```

Cinq groupes, un par ville. `$group` ne trie pas : chez vous, les villes peuvent arriver dans un autre ordre.

3. Ajoutez le tri. `{ commandes: -1 }` trie sur le champ `commandes` calculé par `$group`, du plus grand au

plus petit :

```
db.commandes.aggregate([
  { $match: { statut: "payée" } },
  { $group: { _id: "$client.ville", commandes: { $sum: 1 } } },
  { $sort: { commandes: -1 } }
])
```

Vous devez obtenir

```
[ { _id: 'Fès', commandes: 200 }, { _id: 'Rabat', commandes: 200 },
  { _id: 'Tanger', commandes: 200 }, { _id: 'Marrakech', commandes: 200 },
  { _id: 'Casablanca', commandes: 100 } ]
```

Les quatre villes à 200 commandes peuvent apparaître dans un autre ordre : le tri ne porte que sur le nombre de commandes.

À vous

Écrivez une chaîne d'agrégation pour chaque question. Dans `$group`, les accumulateurs `$sum`, `$min`, `$max` et `$avg` calculent la somme, le minimum, le maximum et la moyenne d'un champ sur les documents du groupe, par exemple `{ $max: "$prix" }`. Pour trier les groupes sur leur clé, triez sur `_id`.

a. Le nombre de produits de chaque catégorie, trié par catégorie.
b. Le nombre de commandes de chaque statut.
c. Pour chaque ville, la date de la première et de la dernière commande payée.
d. Pour chaque catégorie de produits : le stock total, le prix minimal, le prix maximal et le prix moyen.

Exercice 3 : Déplier les tableaux avec $unwind

L'étape $unwind

`{ $unwind: "$lignes" }` remplace chaque document par autant de copies que son tableau `lignes` a d'éléments. Chaque copie est identique au document de départ, sauf le champ `lignes`, qui ne contient plus le tableau mais un seul de ses éléments. Les lignes deviennent ainsi des documents, qu'on peut regrouper, trier ou additionner.

Exemple guidé

1. Dépliez les lignes de la commande 10001. `$project: { lignes: 1 }` ne garde que le champ `lignes`, et `_id`, que `$project` garde toujours sauf si l'on écrit `_id: 0` :

```
db.commandes.aggregate([ { $match: { _id: 10001 } }, { $unwind: "$lignes" },
                         { $project: { lignes: 1 } } ])
```

Vous devez obtenir

```
[ { _id: 10001,
    lignes: { ref: 'P-012', libelle: 'Maison n°12', prix: Decimal128('454.00'), qte: 2 } },
  { _id: 10001,
    lignes: { ref: 'P-029', libelle: 'Sport n°29', prix: Decimal128('83.00'), qte: 3 } } ]
```

La commande 10001, qui a deux lignes, devient deux documents de même `_id` ; dans chacun, `lignes` est un sous-document et non plus un tableau.

2. Comptez les lignes de toutes les commandes payées :

```
db.commandes.aggregate([ { $match: { statut: "payée" } }, { $unwind: "$lignes" },
                         { $count: "lignes" } ])
```

Vous devez obtenir

```
[ { lignes: 1800 } ]
```

Les 900 commandes payées deviennent 1 800 documents, un par ligne de commande.

À vous

Toutes les questions portent sur les commandes payées. Le **chiffre d'affaires** est la somme des montants vendus ; le montant d'une ligne est son prix multiplié par sa quantité, que `{ $multiply: [ a, b ] }` calcule.

a. Le chiffre d'affaires de chaque ville, de la ville qui rapporte le plus à celle qui rapporte le moins.
b. Les cinq produits les plus vendus, en nombre d'exemplaires (la somme des quantités de leurs lignes). Départagez les égalités par la référence.
c. Le chiffre d'affaires de chaque mois, dans l'ordre des mois. Aide : `{ $dateToString: { format: "%Y-%m", date: "$date" } }` écrit la date sous la forme « 2026-01 ».
d. Le nombre moyen de lignes par commande. Aide : `{ $size: "$lignes" }` donne le nombre d'éléments du tableau `lignes`. Faut-il `$unwind` ? Pourquoi ?

Exercice 4 : Calculer dans chaque document

Exemple guidé

Le total d'une commande est la somme, sur toutes ses lignes, du prix multiplié par la quantité. On peut le calculer sans déplier les lignes, avec trois opérateurs :

— `$map` applique un calcul à chaque élément d'un tableau et renvoie le tableau des résultats. `input` désigne le tableau, `as` donne un nom à l'élément en cours (ici `l`), et `in` donne le calcul, où `$$l` (deux dollars) désigne cet élément : `"$$l.prix"` est son prix ;

— `$sum`, appliqué à un tableau, additionne ses éléments ;

— `$set` ajoute au document un champ (ici `total`) sans toucher aux autres.

```
db.commandes.aggregate([
  { $match: { _id: { $in: [ 10001, 10002 ] } } },
  { $set: { total: { $sum: { $map: { input: "$lignes", as: "l",
                                     in: { $multiply: [ "$$l.prix", "$$l.qte" ] } } } } } },
  { $project: { total: 1, nb_lignes: { $size: "$lignes" } } }
])
```

Vous devez obtenir

```
[ { _id: 10001, total: Decimal128('1157.00'), nb_lignes: 2 },
  { _id: 10002, total: Decimal128('3162.00'), nb_lignes: 3 } ]
```

Vérification avec les deux lignes de la commande 10001, affichées à l'exercice 1 :
$454 \times 2 + 83 \times 3 = 908 + 249 = 1\,157$.

À vous

a. Les trois commandes payées au plus gros total : numéro, ville et total. Départagez les égalités par le numéro.
b. Le nombre de commandes payées dont le total dépasse 3 000 DH.
c. Le panier moyen de chaque ville (la moyenne des totaux de ses commandes payées), arrondi à l'unité avec `{ $round: [ valeur, 0 ] }`.
d. Rangez le total dans chaque commande, une fois pour toutes. Pour cela, la méthode `updateMany` accepte, en second argument (la modification), une chaîne d'agrégation, c'est-à-dire un tableau d'étapes entre crochets : chaque document est alors modifié par un calcul sur ses propres champs, par exemple l'étape `$set` de l'exemple guidé. Cette commande modifie la collection `commandes` : chaque commande y gagne un champ `total`. Répondez ensuite de nouveau à la question b avec un simple `countDocuments`.

Exercice 5 : Filtrer les groupes

En SQL, `WHERE` filtre les lignes avant le `GROUP BY`, et `HAVING` filtre les groupes après. Une chaîne d'agrégation n'a pas deux mots-clés : un `$match` placé avant `$group` filtre des documents, comme `WHERE` ; un `$match` placé après `$group` filtre des groupes, comme `HAVING`, et peut porter sur les champs que `$group` a calculés.

a. Les villes dont le chiffre d'affaires des commandes payées dépasse 200 000 DH.
b. Le nombre de produits vendus à plus de 30 exemplaires (commandes payées).
c. Le nombre de clients qui ont dépensé plus de 15 000 DH en commandes payées, puis les trois qui ont dépensé le plus. Utilisez le champ `total` rangé dans chaque commande à l'exercice 4, question d (faites-la d'abord si besoin).

Exercice 6 : Joindre avec $lookup

La catégorie et le stock d'un produit ne sont rangés que dans la collection `produits` : une ligne de commande ne recopie que la référence, le libellé et le prix. Pour les obtenir, il faut une **jointure**, c'est-à-dire associer à chaque ligne le produit qui porte la même référence.

> **L'étape $lookup**
>
> `$lookup` ajoute à chaque document les documents d'une autre collection qui lui correspondent. Elle a quatre paramètres : `from`, la collection où chercher ; `localField`, un champ du document en cours ; `foreignField`, le champ de l'autre collection qui doit lui être égal ; `as`, le nom du champ où ranger les documents trouvés, toujours sous la forme d'un tableau.

Exemple guidé

On cherche, pour chaque ligne de la commande 10001, la catégorie et le stock du produit :

```
db.commandes.aggregate([
  { $match: { _id: 10001 } },
  { $unwind: "$lignes" },
  { $lookup: { from: "produits", localField: "lignes.ref", foreignField: "ref", as: "produit" } },
  { $project: { _id: 0, ref: "$lignes.ref", qte: "$lignes.qte",
                produit: { categorie: 1, stock: 1 } } }
])
```

Vous devez obtenir

```
[ { produit: [ { categorie: 'Maison', stock: 6 } ], ref: 'P-012', qte: 2 },
  { produit: [ { categorie: 'Sport', stock: 27 } ], ref: 'P-029', qte: 3 } ]
```

`produit` est un tableau : `$lookup` y range tous les produits trouvés, ici un seul, puisqu'une référence désigne un seul produit. Dans `$project`, `produit: { categorie: 1, stock: 1 }` ne garde que ces deux champs dans chaque élément du tableau.

À vous

a. Le chiffre d'affaires de chaque catégorie (commandes payées). Aide : après `$lookup`, un `{ $unwind: "$produit" }` transforme le tableau d'un élément en sous-document, dont `"$produit.categorie"` donne la catégorie. Rangez la chaîne dans une variable, pour la réutiliser à la question c : `const chaine = [ … ]`, puis `db.commandes.aggregate(chaine)`.

b. Un produit est en **rupture de stock** quand son champ `stock` vaut 0. Comptez les lignes de commandes payées dont le produit est aujourd'hui en rupture de stock, puis donnez leur nombre pour chaque produit.

c. Mesurez le travail de l'étape `$lookup` dans la chaîne de la question a :

```
db.commandes.explain("executionStats").aggregate(chaine).stages.find(e => e.$lookup)
```

Placé avant `aggregate`, `explain("executionStats")` exécute la chaîne et renvoie, au lieu du résultat, le détail de chaque étape dans le tableau `stages`. La fin de la commande ne garde, dans ce tableau, que l'élément qui décrit l'étape `$lookup`. Relevez quatre compteurs :

— `collectionScans` : le nombre de parcours complets de la collection `produits` ;

— `totalDocsExamined` : le nombre de produits lus ;

— `totalKeysExamined` : le nombre de clés d'index lues ;

— `executionTimeMillisEstimate` : la durée estimée de l'étape, en millisecondes.

Créez ensuite un **index unique** sur le champ `ref` de `produits` : un index qui accélère la recherche par référence, et qui refuse deux produits de même référence.

```
db.produits.createIndex({ ref: 1 }, { unique: true })
```

Relancez la même commande `explain` et comparez les quatre compteurs.

Exercice 7 : Ranger un résultat avec $merge

L'étape $merge

Placée en dernier, `$merge` écrit les documents produits par la chaîne dans une collection (`into`) au lieu de les afficher. Pour chaque document, MongoDB cherche dans cette collection un document de même `_id` : s'il en trouve un, `whenMatched` dit quoi faire (`"replace"` : le remplacer) ; sinon, il insère le document. On range ainsi un résultat coûteux à calculer, pour le relire ensuite avec un simple `find`.

L'accumulateur `$addToSet` construit, pour chaque groupe, le tableau des valeurs distinctes d'un champ, ici les numéros de commande ; `$size` compte ensuite ses éléments.

a. Calculez, pour chaque mois, le chiffre d'affaires et le nombre de commandes payées, et rangez le résultat dans la collection `ventes_par_mois` :

```
db.commandes.aggregate([
  { $match: { statut: "payée" } },
  { $unwind: "$lignes" },
  { $group: { _id: { $dateToString: { format: "%Y-%m", date: "$date" } },
              ca: { $sum: { $multiply: [ "$lignes.prix", "$lignes.qte" ] } },
              commandes: { $addToSet: "$_id" } } },
  { $project: { ca: 1, commandes: { $size: "$commandes" } } },
  { $merge: { into: "ventes_par_mois", whenMatched: "replace" } }
])
```

Qu'affiche la chaîne ? Combien de documents contient la collection `ventes_par_mois` ? Comptez-les avec :

```
db.ventes_par_mois.countDocuments()
```

b. Quel mois a le plus gros chiffre d'affaires ? Répondez avec un simple `find`.
c. Relancez la chaîne de la question a. Combien de documents contient maintenant `ventes_par_mois` ? Pourquoi ?
d. Pourquoi compte-t-on les commandes avec `$addToSet` puis `$size`, et non avec `{ $sum: 1 }` ?

Partie 2 : index et plans d'exécution

Index

Un index est une structure rangée à part de la collection, dans son propre fichier : la liste des valeurs d'un champ, triée, où chaque valeur (une **clé**) est accompagnée de l'adresse du document qui la contient. MongoDB la range dans un arbre B (*B-tree*), qui permet d'aller directement à une clé sans tout lire. Pour chercher les commandes du client 42, MongoDB descend dans l'arbre jusqu'aux clés 42, les lit, puis va chercher les documents désignés. Sans index, il doit lire toute la collection.

Ajoutée à une requête, la méthode `explain("executionStats")` exécute la requête et renvoie son **plan d'exécution** : les étapes que le serveur a enchaînées (champ `queryPlanner.winningPlan`, le plan gagnant, choisi parmi les plans possibles) et des compteurs (champ `executionStats`). Les étapes que vous rencontrerez :

Étape	Ce qu'elle fait
`COLLSCAN`	(*collection scan*) lit tous les documents de la collection, un par un
`IXSCAN`	(*index scan*) lit une portion de l'index : les clés qui correspondent au filtre
`FETCH`	va chercher les documents dont l'index a donné l'adresse
`SORT`	trie en mémoire les documents reçus
`LIMIT`	s'arrête après *n* documents
`PROJECTION_DEFAULT`	ne garde que les champs demandés par la projection
`COUNT`, `COUNT_SCAN`	comptent les documents, ou directement les clés de l'index

Quatre compteurs de `executionStats` sont utiles :

— `nReturned` : le nombre de documents renvoyés ;
— `totalKeysExamined` : le nombre de clés d'index lues ;
— `totalDocsExamined` : le nombre de documents lus ;
— `executionTimeMillis` : la durée d'exécution, en millisecondes.

Une requête efficace lit à peu près autant de clés et de documents qu'elle en renvoie.

Exercice 8 : Chercher avec et sans index

Cette partie utilise la collection `commandes_volume` : 200 000 commandes passées par 5 000 clients (40 commandes chacun), datées du 1er janvier 2025 au 30 septembre 2026.

1. Affichez une commande de `commandes_volume` :

```
db.commandes_volume.findOne()
```

Vous devez obtenir

```
{ _id: 1,
  client: { id: 2920, nom: 'Client 2920', ville: 'Rabat' },
  date: ISODate('2025-01-02T00:00:00.000Z'),
  statut: 'payée',
  total: Decimal128('1157.00'),
  lignes: [
    { ref: 'P-012', prix: Decimal128('454.00'), qte: 2 },
    { ref: 'P-029', prix: Decimal128('83.00'), qte: 3 }
  ] }
```

La structure est celle de `commandes`, sans le libellé dans les lignes, et avec un champ `total` déjà calculé.

2. Définissez une fonction qui résume un plan d'exécution en une ligne. Elle part de la dernière étape du plan gagnant et remonte d'étape en étape (chaque étape reçoit ses documents de son `inputStage`), puis renvoie la liste des étapes et les quatre compteurs. Tapez-la, ou collez-la, une seule fois : elle reste disponible jusqu'à la fermeture de `mongosh`.

```
function resume(plan) {
  const etapes = [];
  for (let p = plan.queryPlanner.winningPlan; p; p = p.inputStage) etapes.push(p.stage);
  const s = plan.executionStats;
  return { etapes: etapes.join(" <- "), renvoyes: s.nReturned, cles: s.totalKeysExamined,
           documents: s.totalDocsExamined, ms: s.executionTimeMillis };
}
```

Vous devez obtenir

`mongosh` répond `[Function: resume]`. Dans les résumés, `FETCH <- IXSCAN` se lit de droite à gauche : `IXSCAN` travaille d'abord et passe ses résultats à `FETCH`.

3. Cherchez les commandes du client 42, sans index :

```
resume(db.commandes_volume.find({ "client.id": 42 }).explain("executionStats"))
```

Vous devez obtenir

```
{ etapes: 'COLLSCAN', renvoyes: 40, cles: 0, documents: 200000, ms: 51 }
```

4. Créez un index sur `client.id`, puis relancez la même recherche :

```
db.commandes_volume.createIndex({ "client.id": 1 })
resume(db.commandes_volume.find({ "client.id": 42 }).explain("executionStats"))
```

Vous devez obtenir

```
client.id_1
{ etapes: 'FETCH <- IXSCAN', renvoyes: 40, cles: 40, documents: 40, ms: 1 }
```

`createIndex` répond par le nom de l'index : le champ, suivi du sens de tri (1 : croissant).

5. Comparez la place occupée par les index et par les documents de la collection :

```
db.commandes_volume.stats().indexSizes
db.commandes_volume.stats().storageSize
```

Vous devez obtenir

```
{ _id_: 2203648, 'client.id_1': 1150976 }
14352384
```

`indexSizes` donne la taille de chaque index, et `storageSize` celle des documents, en octets, telles qu'elles sont rangées sur le disque, c'est-à-dire compressées. Le serveur n'écrit ses fichiers sur le disque qu'une fois par minute : juste après le chargement, les tailles affichées sont plus petites. Attendez une minute avant de les relever. Elles varient aussi d'une machine à l'autre : seul l'ordre de grandeur compte.

a. Combien de documents la base lit-elle pour chaque document renvoyé, avant puis après l'index ?
b. Que signifient les étapes `COLLSCAN`, puis `FETCH <- IXSCAN` ?
c. Quelle place occupe l'index `client.id_1`, comparée à celle des documents ?

Exercice 9 : L'index composé et la règle ESR

Index composé

Un index composé porte sur plusieurs champs. Ses clés sont triées sur le premier champ, puis, à valeur égale, sur le deuxième, et ainsi de suite. Dans l'index `{ "client.id": 1, date: -1 }`, toutes les clés du client 42 se suivent, de la plus récente à la plus ancienne : (42, 30/09/2026), (42, 16/09/2026), (42, 02/09/2026), etc. ; viennent ensuite celles du client 43.

Exemple déroulé : les clés du client 42

Voici une partie de l'index `{ "client.id": 1, date: -1 }`, dans l'ordre où il range ses clés. Chaque clé est accompagnée de l'adresse d'un document, désigné ici par son `_id` :

Rang parmi les clés du client 42	Clé (`client.id`, `date`)	Document désigné
	(41, 05/01/2025)	la dernière clé du client 41
1	(42, 30/09/2026)	commande 114839
2	(42, 16/09/2026)	commande 84839
3	(42, 02/09/2026)	commande 54839
…	…	…
10	(42, 07/05/2026)	commande 29839
11	(42, 03/04/2026)	commande 184839
…	…	…
40	(42, 14/01/2025)	commande 144839
	(43, 25/09/2026)	la première clé du client 43

Pour trouver les 10 commandes les plus récentes du client 42, MongoDB peut donc procéder en trois temps :

1. descendre dans l'arbre jusqu'à la première clé du client 42, (42, 30/09/2026) ;
2. lire les clés suivantes une à une (elles sont déjà de la plus récente à la plus ancienne) et aller chercher le document de chacune ;
3. s'arrêter après la 10e, (42, 07/05/2026) : les 30 autres clés du client ne sont jamais lues, et aucun tri n'est nécessaire.

Les étapes suivantes le vérifient, en comparant avec un index sur `client.id` seul.

Éviter le tri en mémoire

1. Cherchez les 10 commandes les plus récentes du client 42, avec le seul index `client.id_1` de l'exercice 8 :

```
resume(db.commandes_volume.find({ "client.id": 42 }).sort({ date: -1 }).limit(10)
                          .explain("executionStats"))
```

Vous devez obtenir

```
{ etapes: 'SORT <- FETCH <- IXSCAN', renvoyes: 10, cles: 40, documents: 40, ms: 0 }
```

2. Créez un index composé sur le client puis la date, et relancez la commande de l'étape 1 :

```
db.commandes_volume.createIndex({ "client.id": 1, date: -1 })
```

Vous devez obtenir

```
client.id_1_date_-1
{ etapes: 'LIMIT <- FETCH <- IXSCAN', renvoyes: 10, cles: 10, documents: 10, ms: 0 }
```

3. L'index `client.id_1` est-il encore utile ? Supprimez-le, puis affichez le plan gagnant de la recherche de l'exercice 8 :

```
db.commandes_volume.dropIndex("client.id_1")
db.commandes_volume.find({ "client.id": 42 }).explain().queryPlanner.winningPlan
```

Vous devez obtenir

```
{ nIndexesWas: 3, ok: 1 }
{ isCached: false,
  stage: 'FETCH',
  inputStage: {
    stage: 'IXSCAN',
    keyPattern: { 'client.id': 1, date: -1 },
    indexName: 'client.id_1_date_-1',
    isMultiKey: false,
    multiKeyPaths: { 'client.id': [], date: [] },
    isUnique: false, isSparse: false, isPartial: false, indexVersion: 2,
    direction: 'forward',
    indexBounds: { 'client.id': [ '[42, 42]' ], date: [ '[MaxKey, MinKey]' ] } } }
```

`nIndexesWas: 3` : la collection avait trois index avant la suppression (`_id_`, `client.id_1` et le nouveau). Le plan utilise désormais l'index `client.id_1_date_-1` (ligne `indexName`) ; `indexBounds` montre la portion lue : le client 42, toutes dates confondues.

Comparer trois ordres de champs : la règle ESR

Règle ESR

Pour une requête qui combine une égalité, un tri et une plage de valeurs, on range les champs de l'index dans cet ordre : d'abord les champs testés par **É**galité, puis le champ du tri (**S**, de l'anglais *sort*), enfin les champs testés par une plage (**R**, de l'anglais *range*), comme « date postérieure au 1er juin ».

Exemple déroulé : huit commandes

La requête « les 2 plus grosses commandes de Rabat depuis le 1er juin 2026 » teste la ville par égalité (E), trie sur le total (S) et teste la date par une plage (R). On la lance d'abord sur une collection de huit commandes, pour suivre clé par clé ce que lit MongoDB avec chacun des trois ordres possibles.

4. Créez la collection `esr` et trois index qui rangent les mêmes champs dans des ordres différents. L'option `name` donne à chacun un nom qui rappelle l'ordre de ses champs : `R_E_S` commence par la date (R), puis la ville (E), puis le total (S). Lancez ensuite la requête avec chaque index : `hint` impose l'index à utiliser, au lieu de laisser MongoDB choisir.

```
db.esr.insertMany([
  { _id: 1, ville: "Rabat",  total: 900, date: ISODate("2026-05-10") },
  { _id: 2, ville: "Rabat",  total: 800, date: ISODate("2026-07-05") },
  { _id: 3, ville: "Rabat",  total: 700, date: ISODate("2026-06-15") },
  { _id: 4, ville: "Rabat",  total: 600, date: ISODate("2026-09-01") },
  { _id: 5, ville: "Rabat",  total: 500, date: ISODate("2026-08-25") },
  { _id: 6, ville: "Fès",    total: 950, date: ISODate("2026-06-15") },
  { _id: 7, ville: "Fès",    total: 400, date: ISODate("2026-07-05") },
  { _id: 8, ville: "Tanger", total: 850, date: ISODate("2026-08-25") }
])
db.esr.createIndex({ date: 1, ville: 1, total: -1 }, { name: "R_E_S" })
db.esr.createIndex({ ville: 1, date: 1, total: -1 }, { name: "E_R_S" })
db.esr.createIndex({ ville: 1, total: -1, date: 1 }, { name: "E_S_R" })
for (const nom of [ "R_E_S", "E_R_S", "E_S_R" ])
  print(nom, resume(db.esr.find({ ville: "Rabat", date: { $gte: ISODate("2026-06-01") } })
                    .sort({ total: -1 }).limit(2).hint(nom).explain("executionStats")))
```

Vous devez obtenir

```
R_E_S { etapes: 'FETCH <- SORT <- IXSCAN', renvoyes: 2, cles: 7, documents: 2, ms: 0 }
E_R_S { etapes: 'FETCH <- SORT <- IXSCAN', renvoyes: 2, cles: 4, documents: 2, ms: 0 }
E_S_R { etapes: 'LIMIT <- FETCH <- IXSCAN', renvoyes: 2, cles: 3, documents: 2, ms: 0 }
```

Les trois index donnent la même réponse, les commandes 2 (800 DH) et 3 (700 DH), mais en lisant 7, 4 et 3 clés ; les deux premiers trient en mémoire (`SORT`).

5. Le tableau suivant donne les clés de chaque index, dans l'ordre où il les range, et ce que MongoDB fait de chacune : il la lit et la retient, la lit et l'écarte, ou ne la lit pas. Suivez-le colonne par colonne, puis supprimez la collection d'essai : `db.esr.drop()`.

R_E_S : date, ville, total		E_R_S : ville, date, total		E_S_R : ville, total, date	
10/05, Rabat, 900	non lue	Fès, 15/06, 950	non lue	Fès, 950, 15/06	non lue
15/06, Fès, 950	écartée	Fès, 05/07, 400	non lue	Fès, 400, 05/07	non lue
15/06, Rabat, 700	retenue	Rabat, 10/05, 900	non lue	Rabat, 900, 10/05	écartée
05/07, Fès, 400	écartée	Rabat, 15/06, 700	retenue	Rabat, 800, 05/07	retenue
05/07, Rabat, 800	retenue	Rabat, 05/07, 800	retenue	Rabat, 700, 15/06	retenue
25/08, Rabat, 500	retenue	Rabat, 25/08, 500	retenue	Rabat, 600, 01/09	non lue
25/08, Tanger, 850	écartée	Rabat, 01/09, 600	retenue	Rabat, 500, 25/08	non lue
01/09, Rabat, 600	retenue	Tanger, 25/08, 850	non lue	Tanger, 850, 25/08	non lue
7 clés lues ; tri en mémoire		4 clés lues ; tri en mémoire		3 clés lues ; pas de tri	

— `R_E_S` range les clés par date : à partir du 15/06, celles de Rabat sont mêlées à celles des autres villes. MongoDB lit chaque clé d'une autre ville qu'il rencontre, l'écarte et saute à la clé suivante de Rabat : 7 clés lues pour 4 retenues, qu'il doit ensuite trier par total.

— `E_R_S` réunit les clés de Rabat, rangées par date : MongoDB saute directement à (Rabat, 01/06), lit les 4 clés de la plage, puis les trie par total en mémoire avant de garder les 2 premières.

— `E_S_R` réunit les clés de Rabat, rangées du plus gros total au plus petit : MongoDB les lit dans cet ordre, vérifie la date sur chaque clé, écarte celle du 10/05 et s'arrête dès qu'il en a retenu 2. Pas de tri : l'ordre de l'index est celui demandé.

D'où la règle : l'égalité d'abord, pour réunir les clés cherchées ; le tri ensuite, pour qu'elles soient déjà dans l'ordre voulu ; la plage en dernier, vérifiée sur chaque clé pendant la lecture. Avec 8 commandes, l'écart est de quelques clés ; on le mesure maintenant sur 200 000.

Vérifier la règle sur 200 000 commandes

La même requête, sur `commandes_volume` : les 10 plus grosses commandes de Rabat passées depuis le 1er juin 2026.

6. Créez les trois mêmes index sur `commandes_volume`, où la ville est le champ `client.ville` :

```
db.commandes_volume.createIndex({ date: 1, "client.ville": 1, total: -1 }, { name: "R_E_S" })
db.commandes_volume.createIndex({ "client.ville": 1, date: 1, total: -1 }, { name: "E_R_S" })
db.commandes_volume.createIndex({ "client.ville": 1, total: -1, date: 1 }, { name: "E_S_R" })
```

Vous devez obtenir

Chaque commande répond par le nom de son index : `R_E_S`, `E_R_S`, `E_S_R`.

7. Mesurez la requête avec chaque index :

```
resume(db.commandes_volume.find({ "client.ville": "Rabat",
                                  date: { $gte: ISODate("2026-06-01") } })
       .sort({ total: -1 }).limit(10).hint("R_E_S").explain("executionStats"))
```

Recommencez avec `"E_R_S"`, puis `"E_S_R"`, et remplissez le tableau ci-dessous. Une étape `SORT` dans le résumé signifie que MongoDB a trié en mémoire.

8. Relancez la requête sans `hint`, et affichez le plan que MongoDB choisit seul. La première commande range le plan dans la constante `plan`, la seconde affiche le plan gagnant :

```
const plan = db.commandes_volume.find({ "client.ville": "Rabat",
  date: { $gte: ISODate("2026-06-01") } }).sort({ total: -1 }).limit(10).explain()
plan.queryPlanner.winningPlan
```

Quel index MongoDB a-t-il choisi (ligne `indexName`) ?

9. Supprimez les trois index d'essai :

```
db.commandes_volume.dropIndex("R_E_S")
db.commandes_volume.dropIndex("E_R_S")
db.commandes_volume.dropIndex("E_S_R")
```

Index	Étapes	Tri en mémoire ?	Clés lues	Documents lus
R_E_S				
E_R_S				
E_S_R				

La requête couverte

Requête couverte

Une requête est couverte quand son filtre et les champs qu'elle renvoie sont tous dans l'index : MongoDB répond avec l'index seul, sans lire aucun document. Les champs renvoyés se choisissent avec la projection, le second argument de `find` : 1 pour garder un champ, 0 pour le masquer ; `_id` est renvoyé sauf si l'on écrit `_id: 0`.

Une clé de l'index `client.id_1_date_-1`, comme (42, 30/09/2026), ne contient que ces deux valeurs et l'adresse du document : ni `_id`, ni `total`, ni `lignes`. Une requête qui ne demande que `client.id` et `date` peut donc être servie par les seules clés ; dès qu'elle demande un autre champ, il faut lire le document.

10. Comparez ces deux requêtes, qui ne diffèrent que par `_id: 0`. Toutes deux filtrent sur `client.id` et demandent `client.id` et `date`, les deux champs de l'index `client.id_1_date_-1` :

```
resume(db.commandes_volume.find({ "client.id": 42 }, { _id: 0, "client.id": 1, date: 1 })
                          .explain("executionStats"))
resume(db.commandes_volume.find({ "client.id": 42 }, { "client.id": 1, date: 1 })
                          .explain("executionStats"))
```

a. Pourquoi l'étape `SORT` disparaît-elle avec l'index composé (étapes 1 et 2) ?
b. Pourquoi l'index `client.id_1` est-il devenu inutile (étape 3) ?
c. Complétez le tableau. Quel ordre de champs est le meilleur, et pourquoi ? Quel index MongoDB choisit-il sans `hint` ?
d. Combien de documents lit la première requête de l'étape 10 ? La seconde ? Pourquoi ?

Exercice 10 : Diagnostiquer avec le profileur

Profileur

Le profileur est un enregistreur intégré au serveur : il écrit une trace de chaque opération plus lente qu'un seuil (`slowms`, en millisecondes) dans la collection `system.profile` de la base courante. Au niveau 0, il est arrêté (c'est le réglage par défaut) ; au niveau 1, il enregistre les opérations plus lentes que le seuil ; au niveau 2, il les enregistre toutes. Il sert à trouver, pendant qu'une application tourne, les requêtes auxquelles il manque un index.

1. Activez le profileur au niveau 1 : il enregistrera les opérations de plus de 20 ms.

```
db.setProfilingLevel(1, { slowms: 20 })
```

Vous devez obtenir

```
{ was: 0, slowms: 100, sampleRate: 1, ok: 1 }
```

La réponse décrit l'état d'avant : profileur arrêté (`was: 0`) et seuil par défaut de 100 ms. Si le profileur a déjà servi sur votre serveur, ces valeurs peuvent différer.

2. Lancez ces trois requêtes, comme le ferait une application : les 5 dernières commandes annulées de Fès, le nombre de commandes qui contiennent le produit P-100, les 10 dernières commandes du client 42.

```
db.commandes_volume.find({ statut: "annulée",
                           "client.ville": "Fès" }).sort({ date: -1 }).limit(5)
db.commandes_volume.countDocuments({ "lignes.ref": "P-100" })
db.commandes_volume.find({ "client.id": 42 }).sort({ date: -1 }).limit(10)
```

Vous devez obtenir

La première et la troisième affichent leurs commandes, la deuxième affiche `2000`. Ces résultats ne nous intéressent pas ici : ce qui compte est la trace que chaque requête laisse, ou non, dans le profileur.

3. Lisez ce que le profileur a enregistré, de l'opération la plus lente à la plus rapide, puis arrêtez-le (niveau 0) en remettant le seuil par défaut, 100 ms :

```
db.system.profile.find({}, { _id: 0, op: 1, ns: 1, millis: 1, planSummary: 1,
                             docsExamined: 1, nreturned: 1 }).sort({ millis: -1 })
db.setProfilingLevel(0, { slowms: 100 })
```

Vous devez obtenir

```
[ { op: 'query', ns: 'boutique.commandes_volume', docsExamined: 200000, nreturned: 5,
    millis: 63, planSummary: 'COLLSCAN' },
  { op: 'command', ns: 'boutique.commandes_volume', docsExamined: 200000, nreturned: 1,
    millis: 55, planSummary: 'COLLSCAN' } ]
{ was: 1, slowms: 20, sampleRate: 1, ok: 1 }
```

Chaque trace donne le type d'opération (`op`), la collection visée, sous la forme `base.collection` (`ns`), la durée en millisecondes (`millis`), le résumé du plan (`planSummary`), les documents lus et renvoyés. Les deux traces peuvent apparaître dans l'autre ordre : le tri se fait sur la durée.

a. Quelles requêtes le profileur a-t-il enregistrées ? Pourquoi la troisième n'y est-elle pas ?
b. Pourquoi la deuxième apparaît-elle comme une `command`, avec `nreturned: 1` ?
c. Proposez un index pour chacune des deux requêtes lentes. Créez-les, puis vérifiez avec `resume` que les plans ont changé. Pour la première, ajoutez `.explain("executionStats")` à la requête, comme à l'exercice 9. La deuxième ne se prête pas à `explain` : `countDocuments` renvoie un nombre, pas une requête. La méthode `count`, placée après `explain`, fait le même comptage :

```
resume(db.commandes_volume.explain("executionStats").count({ "lignes.ref": "P-100" }))
```

Exercice 11 : Les index spécialisés

Cet exercice revient sur les petites collections de la base `boutique`. Il utilise aussi la fonction `resume` de l'exercice 8 : si vous avez fermé `mongosh`, retapez-la d'abord.

a. **Index unique.** Un index unique refuse deux documents qui ont la même valeur de clé : toute insertion ou modification qui créerait un doublon échoue. L'exercice 6 a créé un index unique sur `produits.ref`. Essayez d'insérer un second produit P-001 :

```
db.produits.insertOne({ ref: "P-001", libelle: "Doublon" })
```

b. **Index TTL** (*time to live*, durée de vie). Un index TTL porte sur un champ de type date, avec l'option `expireAfterSeconds`. Une tâche du serveur passe toutes les 60 secondes et supprime les documents dont cette date, augmentée du délai, est dépassée. Créez une collection de sessions dont les documents expirent 30 minutes (1 800 secondes) après leur dernière activité, et insérez-y une session inactive depuis deux heures et une session active à l'instant :

```
db.sessions.createIndex({ derniere_activite: 1 }, { expireAfterSeconds: 1800 })
db.sessions.insertMany([
  { _id: "s1", client: 8,  derniere_activite: new Date(Date.now() - 2 * 3600 * 1000) },
  { _id: "s2", client: 15, derniere_activite: new Date() }
])
db.sessions.countDocuments()
```

Attendez une minute, puis comptez de nouveau les sessions. Qu'observez-vous ?

c. **Index partiel.** Un index partiel ne contient que les documents qui vérifient un filtre, donné par l'option `partialFilterExpression`. Indexez la date des seules commandes payées, puis comparez deux recherches, l'une avec le statut, l'autre sans :

```
db.commandes.createIndex({ date: -1 }, { partialFilterExpression: { statut: "payée" } })
resume(db.commandes.find({ statut: "payée", date: { $gte: ISODate("2026-09-01") } })
                   .explain("executionStats"))
resume(db.commandes.find({ date: { $gte: ISODate("2026-09-01") } })
                   .explain("executionStats"))
```

d. **Index multiclé.** Un index sur un champ tableau est multiclé : MongoDB y range une clé pour chaque élément du tableau. Indexez le tableau `tags` des produits, cherchez les produits en promotion, puis comptez les clés de chaque index avec `validate`, qui vérifie la collection et ses index (champ `keysPerIndex`) :

```
db.produits.createIndex({ tags: 1 })
resume(db.produits.find({ tags: "promo" }).explain("executionStats"))
db.produits.validate({ full: true }).keysPerIndex
```

Pourquoi l'index `tags_1` a-t-il plus de clés qu'il n'y a de produits ?

Partie 3 : les mêmes outils dans Compass

Avant de commencer

Les libellés cités sont ceux de Compass 1.51 ; ses menus sont en anglais. Selon la version, un bouton peut changer légèrement de nom ou de place.

Exercice 12 : Construire une chaîne d'agrégation

On reconstruit dans Compass la chaîne du chiffre d'affaires par ville (exercice 3, question a), en regardant le résultat de chaque étape.

1. Ouvrez Compass. Si la connexion à `localhost:27017` n'apparaît pas dans la liste de gauche, cliquez sur **Add new connection**, laissez l'adresse proposée (`mongodb://localhost:27017`), puis cliquez sur **Save & Connect**. Avec Docker, cette adresse fonctionne parce que le conteneur publie le port 27017.

2. Dans la liste de gauche, dépliez la base `boutique`, cliquez sur la collection `commandes`, puis sur l'onglet **Aggregations**.

3. Si aucune étape n'est affichée, cliquez sur **Add stage**. Dans la liste déroulante de l'étape, choisissez `$match`, puis remplacez le contenu de l'éditeur par :

   ```
   { statut: "payée" }
   ```

   À droite de l'étape, Compass affiche un échantillon des documents qui en sortent : des commandes au statut `payée`.

4. Ajoutez une étape (**Add stage**), choisissez `$unwind` et écrivez `"$lignes"`. Dans l'échantillon, le champ `lignes` de chaque document ne contient plus un tableau, mais une seule ligne.

5. Ajoutez une étape `$group`. Son échantillon montre des documents de la forme `{ _id: ville, ca: montant }`.

   ```
   { _id: "$client.ville", ca: { $sum: { $multiply: [ "$lignes.prix", "$lignes.qte" ] } } }
   ```

6. Ajoutez une étape `$sort` : `{ ca: -1 }`.

7. Cliquez sur **Run** pour exécuter toute la chaîne sur toute la collection.

   Vous devez obtenir

   Les cinq villes, de Rabat (265 897 DH) à Casablanca (145 575 DH), comme dans `mongosh`.

8. Passez en affichage texte (sélecteur **Stages** / **Text**, en haut de l'éditeur) : la chaîne entière apparaît, telle qu'on l'écrirait dans `mongosh`.

9. Enregistrez-la : **Save**, puis **Save as**, avec le nom « CA par ville ».

10. Cliquez sur **Export Code**, puis choisissez **Python** : Compass écrit le code qui lance la même chaîne depuis un programme Python.

11. Revenez à l'affichage par étapes (**Stages**) et désactivez l'étape `$match` avec son interrupteur, sans la supprimer. Relancez la chaîne (**Run**).

    a. Combien de documents Compass affiche-t-il dans l'échantillon de chaque étape ? Pourquoi n'affiche-t-il pas tout ?
    b. Qu'apporte la construction étape par étape, par rapport à `mongosh` ?
    c. Quel résultat obtenez-vous quand `$match` est désactivée ? Pourquoi Casablanca change-t-elle de place ?

Exercice 13 : Plan d'exécution et index dans Compass

1. Dans la liste de gauche, cliquez sur la collection `commandes_volume`, onglet **Documents**. Tapez ce filtre dans la barre **Filter**, puis cliquez sur **Find** :

```
{ "client.nom": "Client 4242" }
```

2. Cliquez sur **Explain**, à côté de **Find**. Relevez le schéma du plan et le résumé : les compteurs « documents returned », « documents examined » et « index keys examined », et la durée d'exécution. Ce sont les compteurs `nReturned`, `totalDocsExamined` et `totalKeysExamined` de l'exercice 8.
3. Ouvrez l'onglet **Indexes**, puis cliquez sur **Create Index**. Choisissez le champ `client.nom` et le type `1 (asc)`, puis cliquez sur **Create Index**.
4. Revenez à l'onglet **Documents**, cliquez sur **Find**, puis sur **Explain**, avec le même filtre.
5. Dans l'onglet **Indexes**, observez les colonnes **Size** et **Usage** de chaque index.

Vous devez obtenir

	Sans index	Avec l'index `client.nom_1`
plan	`COLLSCAN`	`IXSCAN`, puis `FETCH`
documents returned	40	40
documents examined	200 000	40
index keys examined	0	40
durée	environ 50 ms	environ 1 ms

Le client 4242 a 40 commandes. Sans index sur le nom, MongoDB lit toute la collection pour les trouver ; avec l'index, il lit 40 clés et 40 documents.

a. Quelle commande `mongosh` crée le même index ?
b. Que mesure la colonne **Usage** ? À quoi sert-elle ?
c. Faites la liste des index de `commandes_volume`. Pour chacun, indiquez quelle requête de l'atelier il sert.

Partie 4 : depuis un programme Python (PyMongo)

Une application, par exemple le site de la boutique ou une tâche qui produit chaque nuit un rapport, n'utilise pas `mongosh` : elle envoie ses requêtes depuis son propre code, par un **pilote** (*driver*), une bibliothèque qui parle au serveur. PyMongo est le pilote officiel de MongoDB pour Python. L'atelier 2 l'a installé, l'atelier 3 s'en est servi pour se connecter, lire et modifier des documents ; cette partie ne reprend pas ces bases et lance depuis Python ce que les parties 1 et 2 ont fait dans `mongosh`.

Avant de commencer

— La base `boutique` doit être dans l'état où l'ont laissée les parties 1 à 3 : le champ `total` des commandes (exercice 4, question d), l'index unique `ref_1` des produits (exercice 6) et les index de `commandes_volume` (exercices 9, 10 et 13) servent ici.

— Retrouvez le Python de l'atelier 2. Sous macOS et Linux, activez l'environnement virtuel : `source ~/env-mongo/bin/activate` (l'invite commence par `(env-mongo)`) ; sous Windows, ouvrez PowerShell. Avec Docker, relancez le conteneur Python : `docker start -ai python-mongo`. Il ne voit que le dossier où il a été créé, `atelier02`, sous le nom `/travail` : créez-y un sous-dossier `atelier04` et placez-vous dedans (`cd atelier04` dans le conteneur). Sans Docker, créez un dossier `atelier04` où vous voulez. Copiez-y les trois programmes fournis ; vos propres programmes iront à côté.

— Vérifiez la version de PyMongo ; la commande doit afficher `4.18.2` :

```
python -c "import pymongo; print(pymongo.version)"
```

— Comme ceux des ateliers 2 et 3, les programmes de cette partie lisent l'adresse du serveur après leur nom : sans rien, `mongodb://localhost:27017` (le serveur de votre machine, ou le conteneur `mongo` qui publie le port 27017) ; dans le conteneur `python-mongo`, donnez `mongodb://mongo:27017`, par exemple `python agregation.py mongodb://mongo:27017`.

— N'appelez aucun de vos fichiers `pymongo.py` ni `bson.py` : Python le prendrait pour le pilote, et `from pymongo import MongoClient` échouerait.

De mongosh à Python : ce qui change

Une chaîne d'agrégation, un filtre ou une modification gardent en Python la même structure et les mêmes opérateurs ; seule l'écriture change, parce que ce sont des objets Python :

Dans `mongosh`	En Python	Ce qui change
`{ statut: "payée" }`	`{"statut": "payée"}`	un dictionnaire : les noms de champs entre guillemets
`[ { $match: … } ]`	`[{"$match": …}]`	une liste de dictionnaires ; les opérateurs aussi entre guillemets
`true`, `false`, `null`	`True`, `False`, `None`	les constantes de Python
`ISODate("2026-07-01")`	`datetime(2026, 7, 1)`	module `datetime` ; sans fuseau, PyMongo la prend pour une heure UTC
`Decimal128("3500.00")`	`Decimal128("3500.00")`	module `bson` ; `.to_decimal()` pour calculer en Python
`countDocuments`, `createIndex`	`count_documents`, `create_index`	les noms des méthodes, en minuscules séparées par « _ »
`// commentaire`	`# commentaire`	

Exercice 14 : Lancer une chaîne d'agrégation depuis Python

Curseur

`aggregate(chaine)` envoie toute la chaîne au serveur, qui l'exécute, et renvoie un curseur : un objet que le programme parcourt avec une boucle `for`. Les documents du résultat arrivent par lots, au fil de la boucle ; chacun est un dictionnaire Python. Le calcul se fait dans le serveur : seul le résultat voyage jusqu'au programme.

1. Créez le fichier `agregation.py`. Il se connecte, puis lance la chaîne du chiffre d'affaires par ville (exercice 3, question a), écrite en Python :

```
import sys

from pymongo import MongoClient

ADRESSE = sys.argv[1] if len(sys.argv) > 1 else "mongodb://localhost:27017"
client = MongoClient(ADRESSE)
db = client["boutique"]

chaine = [
    {"$match": {"statut": "payée"}},
    {"$unwind": "$lignes"},
    {"$group": {"_id": "$client.ville",
                "ca": {"$sum": {"$multiply": ["$lignes.prix", "$lignes.qte"]}}}},
    {"$sort": {"ca": -1}},
]
for doc in db.commandes.aggregate(chaine):
    print(doc)
```

Vous devez obtenir

```
{'_id': 'Rabat', 'ca': Decimal128('265897.00')}
{'_id': 'Fès', 'ca': Decimal128('263360.00')}
{'_id': 'Tanger', 'ca': Decimal128('262514.00')}
{'_id': 'Marrakech', 'ca': Decimal128('257201.00')}
{'_id': 'Casablanca', 'ca': Decimal128('145575.00')}
```

Les cinq villes de l'exercice 3, de Rabat à Casablanca, mais affichées comme Python affiche un dictionnaire.

2. Additionnez les chiffres d'affaires. Un `Decimal128` est le nombre décimal de MongoDB : Python sait l'afficher, mais pas calculer avec lui. Essayez `print(doc["ca"] * 2)` dans la boucle ; Python s'arrête sur cette erreur :

```
TypeError: unsupported operand type(s) for *: 'Decimal128' and 'int'
```

La méthode `to_decimal()` convertit un `Decimal128` en `Decimal`, le nombre décimal exact de Python. Ajoutez `from decimal import Decimal` en tête du fichier, puis remplacez la boucle par celle-ci :

```
total = Decimal("0")
for doc in db.commandes.aggregate(chaine):
    ca = doc["ca"].to_decimal()            # Decimal128 -> Decimal
    total += ca
    print(f"{doc['_id']:<12}{ca:>12} DH")
print(f"{'Total':<12}{total:>12} DH")
```

Vous devez obtenir

```
Rabat          265897.00 DH
Fès            263360.00 DH
Tanger         262514.00 DH
Marrakech      257201.00 DH
Casablanca     145575.00 DH
Total         1194547.00 DH
```

Dans `f"…"`, `{ca:>12}` écrit `ca` sur 12 caractères, calé à droite, et `<` cale à gauche. Le total est exact, au centime près.

3. Ajoutez une date et une jointure : le chiffre d'affaires de chaque catégorie depuis le 1er juillet 2026, avec le nombre de lignes vendues. Ajoutez `from datetime import datetime` en tête du fichier, et ces

lignes à la fin :

```
chaine_cat = [
    {"$match": {"statut": "payée", "date": {"$gte": datetime(2026, 7, 1)}}},
    {"$unwind": "$lignes"},
    {"$lookup": {"from": "produits", "localField": "lignes.ref",
                 "foreignField": "ref", "as": "produit"}},
    {"$unwind": "$produit"},
    {"$group": {"_id": "$produit.categorie", "lignes": {"$sum": 1},
                "ca": {"$sum": {"$multiply": ["$lignes.prix", "$lignes.qte"]}}}},
    {"$sort": {"ca": -1}},
]
print()
for doc in db.commandes.aggregate(chaine_cat):
    print(doc["_id"], doc["lignes"], doc["ca"].to_decimal())
```

Vous devez obtenir

Après le tableau de l'étape 2 et une ligne vide :

```
Maison 112 75128.00
Sport 111 73858.00
Informatique 104 72095.00
Téléphonie 83 56633.00
Livres 94 56545.00
```

`datetime(2026, 7, 1)` joue le rôle de `ISODate("2026-07-01")` : la base compte 277 commandes à partir de cette date, avec l'une comme avec l'autre. Le `$lookup` s'écrit comme dans `mongosh`, et profite de l'index `ref_1` de l'exercice 6.

Exercice 15 : Ranger le résultat d'une chaîne

Un résultat se range de deux façons. Le **serveur** peut l'écrire lui-même, avec une dernière étape `$merge` ou `$out` : le résultat ne quitte pas le serveur. Le **programme** peut aussi le lire, le compléter, puis l'écrire avec `insert_many` : c'est utile quand il y ajoute ce que le serveur ne connaît pas, comme un calcul fait en Python ou la date du traitement.

$merge et $out

`$merge` range chaque document du résultat dans la collection cible : il remplace (ou complète) celui de même `_id`, insère les autres, et garde les documents de la cible que le résultat ne contient pas. `$out` remplace toute la collection cible par le résultat : les documents absents du résultat disparaissent.

1. Créez `ecriture.py`. La chaîne calcule, pour chaque ville, le nombre de commandes payées, le chiffre d'affaires et le panier moyen, à partir du champ `total` ; `$merge` la range dans `stats_villes`, que le programme relit ensuite. `sort("ca", -1)` est le `sort({ ca: -1 })` de `mongosh`.

```
import sys
from datetime import datetime, timezone

from pymongo import MongoClient

ADRESSE = sys.argv[1] if len(sys.argv) > 1 else "mongodb://localhost:27017"
client = MongoClient(ADRESSE)
db = client["boutique"]

par_ville = [
    {"$match": {"statut": "payée"}},
    {"$group": {"_id": "$client.ville", "commandes": {"$sum": 1},
                "ca": {"$sum": "$total"}, "panier": {"$avg": "$total"}}},
    {"$set": {"panier": {"$round": ["$panier", 0]}}},
]
reponse = db.commandes.aggregate(par_ville + [
    {"$merge": {"into": "stats_villes", "whenMatched": "replace"}}])
print(list(reponse))
for doc in db.stats_villes.find().sort("ca", -1):
    print(doc)
```

Vous devez obtenir

```
[]
{'_id': 'Rabat', 'commandes': 200, 'ca': Decimal128('265897.00'), 'panier': Decimal128('1329')}
{'_id': 'Fès', 'commandes': 200, 'ca': Decimal128('263360.00'), 'panier': Decimal128('1317')}
{'_id': 'Tanger', 'commandes': 200, 'ca': Decimal128('262514.00'), 'panier': Decimal128('1313')}
{'_id': 'Marrakech', 'commandes': 200, 'ca': Decimal128('257201.00'), 'panier': Decimal128('1286')}
{'_id': 'Casablanca', 'commandes': 100, 'ca': Decimal128('145575.00'), 'panier': Decimal128('1456')}
```

`par_ville + [ … ]` ajoute l'étape `$merge` à la fin de la liste. `aggregate` exécute la chaîne dès l'appel ; le curseur qu'il renvoie est vide (`[]`) : le résultat est dans `stats_villes`. On y retrouve les paniers moyens de l'exercice 4, question c.

2. Le programme écrit maintenant lui-même : il relit `stats_villes`, calcule la part de chaque ville dans le chiffre d'affaires et la date du calcul, puis range ces cinq documents dans `rapport_villes`. `delete_many({})` vide d'abord la collection, pour qu'un second lancement ne crée pas de doublons. Ajoutez à la fin du fichier :

```
villes = list(db.stats_villes.find())
ca_total = sum(v["ca"].to_decimal() for v in villes)
rapport = [{"ville": v["_id"],
            "part_ca": round(float(v["ca"].to_decimal() / ca_total * 100), 1),
            "calcule_le": datetime.now(timezone.utc)}
           for v in villes]
db.rapport_villes.delete_many({})
resultat = db.rapport_villes.insert_many(rapport)
print(len(resultat.inserted_ids), "documents inseres")
for doc in db.rapport_villes.find({}, {"_id": 0, "calcule_le": 0}).sort(
        [("part_ca", -1), ("ville", 1)]):
    print(doc)
print(db.rapport_villes.find_one()["calcule_le"])
```

Vous devez obtenir

Après les lignes de l'étape 1 :

```
5 documents inseres
{'ville': 'Rabat', 'part_ca': 22.3}
{'ville': 'Fès', 'part_ca': 22.0}
{'ville': 'Tanger', 'part_ca': 22.0}
{'ville': 'Marrakech', 'part_ca': 21.5}
{'ville': 'Casablanca', 'part_ca': 12.2}
2026-10-17 14:40:12.842000
```

`inserted_ids` contient les cinq `_id` que PyMongo a donnés aux documents. Le tri sur deux champs s'écrit avec une liste de paires : à égalité de part (Fès et Tanger), la ville départage. La dernière ligne est la date du calcul, différente chez vous : PyMongo renvoie les dates en heure UTC, sans fuseau.

3. Comparez `$out` et `$merge` sur une collection d'essai. La première chaîne range les cinq villes ; les deux suivantes ne produisent qu'un document, Rabat. Le `$match` sur `_id`, placé après `$group`, filtre les groupes (exercice 5). Ajoutez à la fin du fichier :

```
rabat = [{"$match": {"_id": "Rabat"}}]
db.commandes.aggregate(par_ville + [{"$out": "essai_sortie"}])
print("$out, 5 villes   :", db.essai_sortie.count_documents({}))
db.commandes.aggregate(par_ville + rabat + [{"$merge": {"into": "essai_sortie"}}])
print("$merge, Rabat    :", db.essai_sortie.count_documents({}))
db.commandes.aggregate(par_ville + rabat + [{"$out": "essai_sortie"}])
print("$out, Rabat      :", db.essai_sortie.count_documents({}))
```

Vous devez obtenir

```
$out, 5 villes   : 5
$merge, Rabat    : 5
$out, Rabat      : 1
```

`$merge` remplace le document de Rabat et garde les quatre autres ; `$out` remplace toute la collection : il ne reste que Rabat.

Exercice 16 : Lire un plan et mesurer une durée

Mesurer une durée

`time.perf_counter()` lit une horloge précise, en secondes ; seul l'écart entre deux lectures a un sens : c'est la durée écoulée entre elles. Une durée varie d'une exécution à l'autre : la première exécution peut être plus lente, et d'autres programmes se partagent la machine. On mesure donc plusieurs fois, et l'on retient la **médiane**, la valeur du milieu une fois les mesures triées.

Un `find` de PyMongo n'interroge le serveur qu'au moment où l'on lit le premier document. Suivi de `explain()`, il demande à la place le plan d'exécution. La réponse est un dictionnaire qui a les mêmes champs que dans `mongosh`, comme `queryPlanner` et `executionStats` : PyMongo demande le niveau de détail le plus complet, `allPlansExecution`, qui contient celui de `executionStats`.

1. Ouvrez le programme fourni `atelier04-mesure.py`. Il cherche les commandes de 3 500 DH ou plus dans `commandes_volume`, sur le champ `total`, qu'aucun index ne couvre encore. Ses deux fonctions :

```
def resume(plan):
    etapes, index = [], None
    p = plan["queryPlanner"]["winningPlan"]
    while p:                                # on remonte d'etape en etape
        etapes.append(p["stage"])
        index = p.get("indexName", index)
        p = p.get("inputStage")
    s = plan["executionStats"]
    return {"etapes": " <- ".join(etapes), "index": index, "renvoyes": s["nReturned"],
            "cles": s["totalKeysExamined"], "documents": s["totalDocsExamined"],
            "ms": s["executionTimeMillis"]}


def chrono(fois=5):
    durees = []
    for _ in range(fois):
        debut = time.perf_counter()
        docs = list(col.find(FILTRE))     # list() lit tous les documents du curseur
        durees.append(round((time.perf_counter() - debut) * 1000, 1))
    return len(docs), durees
```

`resume` fait en Python ce que faisait la fonction de l'exercice 8, et note en plus le nom de l'index utilisé ; `p.get("inputStage")` vaut `None` sur la dernière étape, ce qui arrête la boucle. `chrono` lance la requête cinq fois et renvoie les durées, en millisecondes. La fin du programme :

```
if "total_1" in col.index_information():
    col.drop_index("total_1")

print("Sans index :", resume(col.find(FILTRE).explain()))
n, durees = chrono()
print(f"  {n} documents ; durees (ms) : {durees} ; mediane : {statistics.median(durees)}")

print("create_index :", col.create_index("total"))
print("Avec index :", resume(col.find(FILTRE).explain()))
n, durees = chrono()
print(f"  {n} documents ; durees (ms) : {durees} ; mediane : {statistics.median(durees)}")
```

2. Lancez-le : `python atelier04-mesure.py`. Dans le conteneur `python-mongo`, ajoutez l'adresse du serveur, `mongodb://mongo:27017`, après le nom du programme.

Vous devez obtenir

```
Sans index : {'etapes': 'COLLSCAN', 'index': None, 'renvoyes': 666, 'cles': 0,
              'documents': 200000, 'ms': 44}
  666 documents ; durees (ms) : [45.7, 45.0, 55.7, 52.8, 45.4] ; mediane : 45.7
create_index : total_1
Avec index : {'etapes': 'FETCH <- IXSCAN', 'index': 'total_1', 'renvoyes': 666, 'cles': 666,
              'documents': 666, 'ms': 1}
  666 documents ; durees (ms) : [2.8, 3.0, 2.9, 2.8, 2.6] ; mediane : 2.8
```

Les durées changent d'une machine et d'un lancement à l'autre ; les nombres de clés et de documents lus, non.

a. Combien de fois la requête est-elle plus rapide avec l'index ? Combien de documents lit-elle pour chaque document renvoyé, avant et après ?
b. Avec l'index, le serveur annonce 1 ms (`executionTimeMillis`), le programme mesure 2,8 ms. D'où vient l'écart ?
c. Relancez le programme. Pourquoi la première mesure est-elle de nouveau faite sans index ?

Exercice 17 : Créer, lister et supprimer des index

`create_index` reçoit le nom d'un champ (index simple, croissant) ou une liste de paires (champ, sens) : une liste, parce que l'ordre des champs d'un index composé compte. Les options s'écrivent comme des arguments nommés (`unique=True`, `name="E_S_R"`, `expireAfterSeconds=1800`…) ; la méthode renvoie le nom de l'index. `list_indexes()` renvoie un curseur sur la description de chaque index ; `drop_index(nom)` en supprime un.

1. Créez `index.py`. Copiez-y les lignes du début de `ecriture.py`, jusqu'à `db = client["boutique"]`, et la fonction `resume` de `atelier04-mesure.py`, puis listez les index de `commandes_volume` :

```
col = db.commandes_volume
for ix in col.list_indexes():
    print(ix["name"], dict(ix["key"]))
```

Vous devez obtenir

```
_id_ {'_id': 1}
client.id_1_date_-1 {'client.id': 1, 'date': -1}
statut_1_client.ville_1_date_-1 {'statut': 1, 'client.ville': 1, 'date': -1}
lignes.ref_1 {'lignes.ref': 1}
client.nom_1 {'client.nom': 1}
total_1 {'total': 1}
```

Les index des exercices 9, 10 et 13, et `total_1`, créé à l'exercice 16.

2. Créez un index composé dans l'ordre E, S, R pour la requête « les 5 plus grosses commandes payées de Fès depuis le 1er juillet 2026 » : égalité sur le statut et la ville, tri sur le total, plage sur la date. Ajoutez `from datetime import datetime` en tête du fichier, et à la fin :

```
filtre = {"statut": "payée", "client.ville": "Fès", "date": {"$gte": datetime(2026, 7, 1)}}
print(resume(col.find(filtre).sort("total", -1).limit(5).explain()))
nom = col.create_index([("statut", 1), ("client.ville", 1), ("total", -1), ("date", 1)])
print(nom)
print(resume(col.find(filtre).sort("total", -1).limit(5).explain()))
for doc in col.find(filtre, {"_id": 1, "total": 1, "date": 1}).sort("total", -1).limit(5):
    print(doc)
```

Vous devez obtenir

```
{'etapes': 'LIMIT <- FETCH <- IXSCAN', 'index': 'total_1', 'renvoyes': 5, 'cles': 21,
 'documents': 21, 'ms': 0}
statut_1_client.ville_1_total_-1_date_1
{'etapes': 'LIMIT <- FETCH <- IXSCAN', 'index': 'statut_1_client.ville_1_total_-1_date_1',
 'renvoyes': 5, 'cles': 6, 'documents': 5, 'ms': 2}
{'_id': 57329, 'date': datetime.datetime(2026, 7, 2, 0, 0), 'total': Decimal128('3669.00')}
{'_id': 6929, 'date': datetime.datetime(2026, 7, 4, 0, 0), 'total': Decimal128('3669.00')}
{'_id': 198329, 'date': datetime.datetime(2026, 7, 4, 0, 0), 'total': Decimal128('3669.00')}
{'_id': 147929, 'date': datetime.datetime(2026, 7, 6, 0, 0), 'total': Decimal128('3669.00')}
{'_id': 97529, 'date': datetime.datetime(2026, 7, 8, 0, 0), 'total': Decimal128('3669.00')}
```

Avant, MongoDB a choisi `total_1` : il lit les commandes de la plus chère à la moins chère et vérifie sur chaque document le statut, la ville et la date ; il lui faut 21 documents pour en trouver 5. Avec l'index E, S, R, ces trois tests se font sur les clés : 6 clés lues, dont une écartée (une commande de 3 669 DH du 2 janvier 2025), et seuls les 5 documents retenus sont lus. Sans l'index `total_1` de l'exercice 16, MongoDB aurait pris `statut_1_client.ville_1_date_-1` : 5 189 clés et 5 189 documents lus, puis un tri en mémoire.

3. Créez un index unique sur le champ `ville` de `rapport_villes` (exercice 15), puis essayez d'y insérer une seconde fois Rabat. L'erreur `DuplicateKeyError` se rattrape avec `try` et `except` ; son attribut `details` contient la réponse du serveur. Ajoutez `from pymongo.errors import DuplicateKeyError` en tête du fichier, et à la fin :

```
print(db.rapport_villes.create_index("ville", unique=True))
try:
    db.rapport_villes.insert_one({"ville": "Rabat", "part_ca": 0.0})
except DuplicateKeyError as erreur:
    print("Refus :", erreur.details["keyValue"])
```

Vous devez obtenir

```
ville_1
Refus : {'ville': 'Rabat'}
```

Sans `try`, l'erreur arrêterait le programme. Comme `ecriture.py` vide `rapport_villes` avec `delete_many`, qui garde les index, il peut toujours être relancé : il n'insère jamais deux fois la même ville.

4. L'index E, S, R ne servait qu'à l'essai ; chaque insertion dans `commandes_volume` devrait pourtant le mettre à jour. Supprimez-le, puis vérifiez la liste des noms :

```
col.drop_index(nom)
print(sorted(col.index_information()))
```

Vous devez obtenir

```
['_id_', 'client.id_1_date_-1', 'client.nom_1', 'lignes.ref_1',
 'statut_1_client.ville_1_date_-1', 'total_1']
```

Le programme se relance sans erreur : `create_index` ne fait rien si l'index existe déjà, et l'étape 4 supprime celui de l'étape 2.

Exercice 18 : Écrire en masse

Aller-retour

Chaque appel à `insert_one` ou `update_one` est un **aller-retour** (*round trip*) : le programme envoie une commande au serveur, puis attend sa réponse avant de continuer. `insert_many` et `bulk_write` regroupent de nombreuses écritures dans une seule commande, dans la limite de 100 000 opérations et de 48 Mo par envoi (champs `maxWriteBatchSize` et `maxMessageSizeBytes` de la commande `hello` du serveur).

`bulk_write` reçoit une liste d'opérations, de natures différentes, sur une même collection. Chaque opération est un objet : `InsertOne(document)`, `UpdateOne` ou `UpdateMany` (avec un filtre et une modification), `ReplaceOne`, `DeleteOne`, `DeleteMany`. PyMongo regroupe les opérations successives de même nature en une seule commande.

1. Ouvrez le programme fourni `atelier04-masse.py`. Il enregistre 10 000 ventes de caisse dans la collection `ventes_jour`, puis en annule une sur dix, de trois façons. Un **écouteur** (`Compteur`), branché sur le client, compte les commandes envoyées au serveur. Les trois façons d'écrire :

```python
def une_par_une():
    for doc in nouvelles:
        ventes.insert_one(doc)
    for i in annulees:
        ventes.update_one({"_id": i}, annuler)


def groupees():
    ventes.insert_many(nouvelles)
    ventes.update_many({"_id": {"$in": annulees}}, annuler)


def en_masse():
    operations = [InsertOne(doc) for doc in nouvelles]
    operations += [UpdateOne({"_id": i}, annuler) for i in annulees]
    return ventes.bulk_write(operations)
```

`nouvelles` est la liste des 10 000 ventes, `annulees` celle des 1 000 numéros à annuler (10, 20, …, 10 000), et `annuler` la modification `{"$set": {"statut": "annulée"}}`. Avant chaque façon, la fonction `mesurer` vide la collection, remet le compteur à zéro et déclenche `time.perf_counter()`.

2. Lancez-le : `python atelier04-masse.py`, suivi de l'adresse du serveur dans le conteneur `python-mongo`.

Vous devez obtenir

```
une par une                 1312 ms   11000 commandes
insert_many, update_many      30 ms       2 commandes
bulk_write                    34 ms       2 commandes
bulk_write : 10000 insertions, 1000 modifications
ventes annulees : 1000
```

a. Combien de fois les écritures groupées sont-elles plus rapides ? Combien dure, en moyenne, un aller-retour de la première façon ?
b. Pourquoi `bulk_write` n'envoie-t-il que deux commandes, pour 11 000 opérations ?
c. Le serveur tourne ici sur la même machine que le programme. Qu'en serait-il avec un serveur distant ?

Exercice 19 : Un programme complet : le rapport des ventes

Le programme fourni `atelier04-rapport.py` assemble ce que font les exercices 14 à 18, comme une vraie tâche planifiée : il fait calculer par le serveur le chiffre d'affaires de chaque mois, le range dans la collection `rapport_mensuel` sans jamais créer de doublon, puis le relit et l'affiche. On peut lui donner une ville et l'adresse du serveur, dans n'importe quel ordre : `python atelier04-rapport.py Rabat` ; dans le conteneur `python-mongo`, `python atelier04-rapport.py mongodb://mongo:27017 Rabat`.

1. Lisez la première partie. `arguments` lit la ligne de commande : un argument qui commence par `mongodb://` est l'adresse du serveur, l'autre la ville. `dh` écrit un montant à la française. La chaîne ne regarde qu'une ville si on lui en donne une, et regroupe les commandes payées par mois, avec le `$dateToString` de l'exercice 3.

```
import sys
from datetime import datetime, timezone
from decimal import Decimal

from pymongo import MongoClient, UpdateOne
from pymongo.errors import ConnectionFailure


def arguments():
    """L'adresse du serveur et la ville, lues sur la ligne de commande."""
    adresse, ville = "mongodb://localhost:27017", None
    for arg in sys.argv[1:]:
        if arg.startswith("mongodb://"):
            adresse = arg
        else:
            ville = arg
    return adresse, ville


def dh(montant):
    """Ecrit un montant comme en francais : 265 897,00."""
    return f"{montant:,.2f}".replace(",", " ").replace(".", ",")


def chaine_rapport(ville):
    """La chaine d'agregation : commandes payees, regroupees par mois."""
    filtre = {"statut": "payée"}
    if ville:
        filtre["client.ville"] = ville
    return [
        {"$match": filtre},
        {"$group": {"_id": {"$dateToString": {"format": "%Y-%m", "date": "$date"}},
                    "commandes": {"$sum": 1},
                    "ca": {"$sum": "$total"}}},       # total : exercice 4, question d
        {"$sort": {"_id": 1}},
    ]
```

2. Lisez le début de la fonction `main`. `serverSelectionTimeoutMS=3000` limite à 3 secondes l'attente d'un serveur absent (30 par défaut) ; la commande `ping` vérifie la connexion avant tout travail. L'écriture est un `bulk_write` de `UpdateOne` avec `upsert=True` : pour chaque mois, le document (ville, mois) est modifié s'il existe, créé sinon. Un index unique sur (`portee`, `mois`) garantit qu'il n'y en a jamais deux.

```
def main():
    adresse, ville = arguments()
    portee = ville or "toutes"
    client = MongoClient(adresse, serverSelectionTimeoutMS=3000)
    try:
        client.admin.command("ping")
    except ConnectionFailure:
        sys.exit(f"Serveur MongoDB injoignable a l'adresse {adresse}")
    db = client["boutique"]

    # 1. Lire : le serveur calcule, le programme ne recoit que le resultat
    mois = list(db.commandes.aggregate(chaine_rapport(ville)))
    if not mois:
        sys.exit(f"Aucune commande payee pour la ville {ville}")

    # 2. Ecrire : un UpdateOne avec upsert par mois, tous envoyes en une fois
    db.rapport_mensuel.create_index([("portee", 1), ("mois", 1)], unique=True)
    maintenant = datetime.now(timezone.utc)
    operations = [UpdateOne({"portee": portee, "mois": m["_id"]},
                            {"$set": {"commandes": m["commandes"], "ca": m["ca"],
                                      "calcule_le": maintenant}},
                            upsert=True)
                  for m in mois]
    res = db.rapport_mensuel.bulk_write(operations)
    print(f"rapport_mensuel : {res.upserted_count} insertions, "
          f"{res.modified_count} modifications")
```

3. Lisez la fin : le programme relit le rapport et l'affiche, avec une ligne de total. Les montants sont convertis en `Decimal` pour calculer le panier moyen.

```
    # 3. Relire et afficher
    print(f"\nCommandes payees, ville : {portee}")
    print(f"{'Mois':<9}{'Commandes':>10}{'CA (DH)':>15}{'Panier moyen':>15}")
    total_ca, total_cmd = Decimal("0"), 0
    for r in db.rapport_mensuel.find({"portee": portee}).sort("mois", 1):
        ca = r["ca"].to_decimal()                    # Decimal128 -> Decimal
        total_ca += ca
        total_cmd += r["commandes"]
        print(f"{r['mois']:<9}{r['commandes']:>10}{dh(ca):>15}{dh(ca / r['commandes']):>15}")
    print(f"{'Total':<9}{total_cmd:>10}{dh(total_ca):>15}{dh(total_ca / total_cmd):>15}")


if __name__ == "__main__":
    main()
```

4. Lancez-le sans argument : `python atelier04-rapport.py`.

Vous devez obtenir

```
rapport_mensuel : 9 insertions, 0 modifications

Commandes payees, ville : toutes
Mois      Commandes        CA (DH)   Panier moyen
2026-01         108     144 148,00       1 334,70
2026-02         104     135 240,00       1 300,38
2026-03         112     148 976,00       1 330,14
2026-04         108     146 368,00       1 355,26
2026-05         108     140 408,00       1 300,07
2026-06         108     145 148,00       1 343,96
2026-07          93     123 762,00       1 330,77
2026-08          84     114 292,00       1 360,62
2026-09          75      96 205,00       1 282,73
Total           900   1 194 547,00       1 327,27
```

On retrouve le contenu de `ventes_par_mois` (exercice 7) et le chiffre d'affaires total de l'exercice 14.

5. Lancez-le une deuxième fois, puis avec une ville : `python atelier04-rapport.py Casablanca`, puis avec une ville absente des données : `python atelier04-rapport.py Agadir`.

Vous devez obtenir

Les premières lignes des trois lancements :

```
rapport_mensuel : 0 insertions, 9 modifications
rapport_mensuel : 9 insertions, 0 modifications
Aucune commande payee pour la ville Agadir
```

Le deuxième lancement affiche le même tableau. Pour Casablanca, le tableau compte 100 commandes pour 145 575,00 DH, soit un panier moyen de 1 455,75 DH.

Exercice 20 : À vous : programmes à écrire seul

Chaque programme commence par les lignes de connexion des exercices précédents.

a. **Les meilleurs clients.** Écrivez `clients.py`, qui affiche les 5 clients qui ont le plus dépensé en commandes payées dans `commandes_volume` : nom, ville, nombre de commandes et dépense, écrite avec la fonction `dh` de l'exercice 19. Départagez les égalités par le numéro du client. Aide : l'accumulateur `$first` garde la valeur du premier document du groupe, par exemple `{"$first": "$client.nom"}`.

b. **Un rapport rejouable.** Après les lancements de l'exercice 19, combien de documents contient `rapport_mensuel` ? Comptez-les par portée avec l'étape `$sortByCount`, qui regroupe, compte et trie en une fois : `[{"$sortByCount": "$portee"}]`. Pourquoi le deuxième lancement annonce-t-il 9 modifications et aucune insertion ? Que se passerait-il si le programme écrivait avec des `InsertOne` au lieu des `UpdateOne` avec `upsert=True` ? Essayez sur une copie du programme.

c. **Avec et sans index, sans rien supprimer.** Mesurez, comme à l'exercice 16, le nombre de commandes de `commandes_volume` qui contiennent le produit P-100, compté par `count_documents`. L'argument nommé `hint` impose le plan : une fois `hint="lignes.ref_1"`, une fois `hint=[("$natural", 1)]`, qui impose de lire la collection dans l'ordre où elle est rangée, sans index. Quel est l'écart ?

d. **Une erreur dans un lot.** La collection `ventes_jour` (exercice 18) contient les ventes 1 à 10 000. Envoyez avec `bulk_write` trois `InsertOne` : les `_id` 10001, 5 (qui existe déjà) et 10002, d'abord avec `ordered=True` (la valeur par défaut), puis avec `ordered=False`. Rattrapez l'erreur `BulkWriteError` (module `pymongo.errors`) et affichez `erreur.details["nInserted"]` et le message de chaque erreur (`erreur.details["writeErrors"]`). Quelles ventes sont insérées dans chaque cas ? Supprimez-les ensuite (`_id` supérieur à 10 000).

e. **Des tranches de montant.** Avec l'étape `$bucket`, qui range les documents dans des tranches de

valeurs, répartissez les 200 000 commandes de `commandes_volume` par tranche de `total` (de 0 à 1 000, de 1 000 à 2 000, de 2 000 à 3 000, de 3 000 à 4 000 DH), avec pour chaque tranche le nombre de commandes et le chiffre d'affaires. Rangez le résultat dans la collection `tranches` avec `$out`, puis affichez-la.

Partie 5 (pour aller plus loin) : ce que le cours résume

Le chapitre 3 du cours (chapitre 4 dans la version complète) présente l'agrégation et les index en quelques transparents. Cette partie fait manipuler ce qu'il ne fait que résumer : comment un index est rangé (exercice 21), ce qu'il coûte aux écritures (exercice 22), comment lire un plan en détail (exercice 23), les index texte, géographiques, hachés et génériques (exercice 24), et, pour finir, le choix d'un index pour quatre requêtes (exercice 25). Chaque exercice commence par une explication courte, « La notion ».

Avant de commencer

Les résultats de cette partie partent des données de départ : rechargez les deux scripts (le second prend jusqu'à une minute), puis rouvrez `mongosh` sur la base `boutique` et retapez la fonction `resume` de l'exercice 8.

```
mongosh --file donnees-boutique.js
mongosh --file donnees-volume.js
```

Avec Docker : `docker exec -it mongo mongosh --file /tmp/donnees-boutique.js`, de même pour `donnees-volume.js` (copié d'abord dans le conteneur avec `docker cp`).

Pour la culture : d'où vient la chaîne d'agrégation

En 2009, MongoDB regroupe les documents avec *map-reduce* : le développeur écrit en JavaScript une fonction `map`, qui émet des paires (clé, valeur) pour chaque document, et une fonction `reduce`, qui combine les valeurs d'une même clé. C'est lent et difficile à mettre au point. En 2012, la version 2.2 introduit la chaîne d'agrégation, dont les étapes sont programmées dans le serveur et dont la première peut utiliser un index ; suivent `$lookup` (3.2, 2015), `$bucket` et `$facet` (3.4, 2016), `$merge` (4.2, 2019). En 2021, la version 5.0 déclare *map-reduce* obsolète.
Un `$lookup` lance une recherche par document. S'il revient à chaque calcul et que l'information change rarement (la catégorie d'un produit), on la recopie dans le document qui la lit, comme les lignes des commandes recopient déjà le libellé et le prix : c'est une **référence étendue**. Plus de jointure, mais une copie à mettre à jour si la valeur change.

Exercice 21 : Comment un index est rangé

La notion

Un index est rangé dans son propre fichier, à côté de celui de la collection. Il contient une entrée par document : la clé et le **RecordId**, le numéro du document dans le fichier de la collection. Les entrées sont triées et rangées en **B-arbre**, dans des pages de 16 ko au plus sur le disque : une page racine, parfois des pages intermédiaires, et des feuilles. Avec environ 500 clés par page, 2 niveaux couvrent $500 \times 500 = 250\,000$ clés, et 3 niveaux 125 millions. `IXSCAN` descend de la racine jusqu'à la première clé cherchée et lit les feuilles voisines ; `FETCH` lit ensuite chaque document par son RecordId.

Exemple guidé

1. Affichez le RecordId des premières commandes :

```
db.commandes.find({}, { _id: 1 }).limit(3).showRecordId()
```

Vous devez obtenir

```
[ { _id: 10001, '$recordId': Long('1') },
  { _id: 10002, '$recordId': Long('2') },
  { _id: 10003, '$recordId': Long('3') } ]
```

2. Créez un index sur la ville, puis demandez quel fichier range la collection, et lequel range l'index (option `indexDetails`) :

```
db.commandes.createIndex({ "client.ville": 1 })
db.commandes.stats().wiredTiger.uri
db.commandes.stats({ indexDetails: true }).indexDetails["client.ville_1"].uri
```

Vous devez obtenir

```
client.ville_1
statistics:table:collection-24-2880067213145443894
statistics:table:index-28-2880067213145443894
```

Deux fichiers différents (les numéros changent d'une installation à l'autre).

3. Indexez le client des 200 000 commandes, forcez l'écriture des fichiers sur le disque avec `fsync` (sinon, attendez une minute), puis relevez la place des index et les bornes lues pour le client 42 :

```
db.commandes_volume.createIndex({ "client.id": 1 })
db.adminCommand({ fsync: 1 }).ok
db.commandes_volume.stats().indexSizes
const plan42 = db.commandes_volume.find({ "client.id": 42 }).explain()
plan42.queryPlanner.winningPlan.inputStage.indexBounds
```

Vous devez obtenir

```
client.id_1
1
{ _id_: 2207744, 'client.id_1': 1150976 }
{ 'client.id': [ '[42, 42]' ] }
```

Les tailles varient un peu d'une machine à l'autre.

À vous

a. Pourquoi l'étape `FETCH` a-t-elle besoin du RecordId ?
b. Combien de niveaux le B-arbre de l'index `client.id_1` (200 000 clés) a-t-il, avec environ 500 clés par page ? Combien de pages faut-il lire pour trouver les 40 commandes du client 42 ?
c. Combien d'octets l'index `client.id_1` occupe-t-il par clé, en moyenne ?
d. Que signifie `'[42, 42]'` dans `indexBounds` ?

Exercice 22 : Ce que coûte un index à l'écriture

La notion

Chaque insertion ajoute une clé dans **chaque** index de la collection, chaque suppression en retire une, et chaque modification d'un champ indexé déplace sa clé. Un index accélère donc les lectures, mais ralentit les écritures et occupe de la place, sur le disque et dans la mémoire du serveur. On n'indexe que les champs souvent cherchés ou triés.

Exemple guidé

1. Le profileur (exercice 10), au niveau 2, note chaque opération ; pour une insertion, il donne le nombre de clés d'index écrites (`keysInserted`). Insérez une commande dans une collection neuve, ajoutez cinq index, puis insérez-en une seconde :

```
db.setProfilingLevel(2)
db.essai.insertOne({ _id: 1, client: { id: 1, ville: "Rabat" }, date: new Date(),
                     statut: "payée", total: 10 })
db.essai.createIndex({ "client.id": 1 }); db.essai.createIndex({ "client.ville": 1 })
db.essai.createIndex({ date: 1 }); db.essai.createIndex({ statut: 1 })
db.essai.createIndex({ total: -1 })
db.essai.insertOne({ _id: 2, client: { id: 1, ville: "Rabat" }, date: new Date(),
                     statut: "payée", total: 10 })
db.system.profile.find({ op: "insert", ns: "boutique.essai" },
                       { _id: 0, ninserted: 1, keysInserted: 1 })
db.setProfilingLevel(0)
```

Vous devez obtenir

```
[ { ninserted: 1, keysInserted: 1 },
  { ninserted: 1, keysInserted: 6 } ]
```

2. Mesurez la copie des 200 000 commandes dans une collection vide, d'abord avec le seul index `_id`, puis avec cinq index de plus. `Date.now()` donne l'heure en millisecondes ; la dernière expression de chaque ligne, la durée, est affichée.

```
db.copie.drop()
const t1 = Date.now(); db.commandes_volume.aggregate([ { $merge: "copie" } ]); Date.now() - t1
db.copie.drop()
db.copie.createIndex({ "client.id": 1 }); db.copie.createIndex({ "client.ville": 1 })
db.copie.createIndex({ date: 1 }); db.copie.createIndex({ statut: 1 })
db.copie.createIndex({ total: -1 })
const t2 = Date.now(); db.commandes_volume.aggregate([ { $merge: "copie" } ]); Date.now() - t2
db.adminCommand({ fsync: 1 }).ok
db.copie.stats().storageSize
db.copie.totalIndexSize()
```

Vous devez obtenir

```
3715
...
7251
1
14483456
9400320
```

Les durées dépendent de la machine et varient beaucoup d'un essai à l'autre : la première copie est souvent ralentie, le temps que le serveur charge les données en mémoire. Les tailles, elles, changent peu.

À vous

a. Pourquoi la seconde insertion de l'étape 1 écrit-elle 6 clés ?
b. Relancez deux ou trois fois les deux copies de l'étape 2 : quel rapport de durée obtenez-vous ?
c. Quelle place les index de `copie` occupent-ils, comparée à celle des documents ? Pourquoi cette place compte-t-elle pour la vitesse ?
d. Supprimez les collections d'essai : `db.essai.drop()`, `db.copie.drop()`.

Exercice 23 : Lire un plan en détail

La notion

Quand plusieurs index conviennent, MongoDB essaie chaque plan candidat un court instant et garde le plus rapide (`winningPlan`) ; les autres sont dans `rejectedPlans`. Une étape `SORT` trie en mémoire, dans une limite de 100 Mo (`memLimit` : 104 857 600 octets) ; au-delà, elle écrit des fichiers temporaires sur le disque, ce qui est lent. Une expression régulière ancrée au début (`/^Livres/`) devient une plage de clés ; non ancrée, ou insensible à la casse (`/i`), elle oblige à tester toutes les clés.

Exemple guidé

1. Ajoutez un index sur le statut, puis demandez le plan d'une recherche qui pourrait utiliser l'index de la ville (exercice 21) ou celui du statut. Le niveau `"allPlansExecution"` détaille aussi les plans écartés.

```
db.commandes.createIndex({ statut: 1 })
const q = db.commandes.find({ statut: "payée", "client.ville": "Rabat" })
const e = q.explain("allPlansExecution")
e.queryPlanner.winningPlan.inputStage.indexName
e.queryPlanner.rejectedPlans.length
```

Vous devez obtenir

```
statut_1
client.ville_1
2
```

L'index de la ville a gagné ; deux plans ont été écartés : celui du statut, et un plan qui croise les deux index.

2. Comparez les deux index en les imposant avec `hint` :

```
resume(db.commandes.find({ statut: "payée", "client.ville": "Rabat" })
                   .hint("client.ville_1").explain("executionStats"))
resume(db.commandes.find({ statut: "payée", "client.ville": "Rabat" })
                   .hint("statut_1").explain("executionStats"))
```

3. Triez les 200 000 commandes par total, sans index sur `total` :

```
db.commandes_volume.find({}).sort({ total: -1 }).limit(5).explain().queryPlanner.winningPlan
```

Vous devez obtenir

```
{ isCached: false, stage: 'SORT', sortPattern: { total: -1 }, memLimit: 104857600,
  limitAmount: 5, type: 'simple', inputStage: { stage: 'COLLSCAN', direction: 'forward' } }
```

4. Indexez le libellé des produits, puis comparez trois expressions régulières :

```
db.produits.createIndex({ libelle: 1 })
resume(db.produits.find({ libelle: /^Livres/ }).explain("executionStats"))
const planL = db.produits.find({ libelle: /^Livres/ }).explain()
planL.queryPlanner.winningPlan.inputStage.indexBounds
resume(db.produits.find({ libelle: /n°12/ }).explain("executionStats"))
resume(db.produits.find({ libelle: /^livres/i }).explain("executionStats"))
```

À vous

a. Combien de clés et de documents chaque index de l'étape 2 lit-il pour les 200 commandes de Rabat ? Pourquoi MongoDB a-t-il préféré l'index de la ville ?
b. À l'étape 3, que signifient `memLimit` et `limitAmount` ? Quel index supprimerait l'étape `SORT` ?
c. À l'étape 4, combien de clés chaque expression lit-elle ? Lisez les bornes de `/^Livres/` : pourquoi « Livret » ?

Exercice 24 : Les index texte, géographiques, hachés et génériques

La notion

Un index **texte** découpe les textes en mots et sert l'opérateur `$text` ; une collection n'en a qu'un. Un index **2dsphere** range des positions sur la sphère terrestre, écrites en GeoJSON (`[ longitude, latitude ]`), et sert les recherches par distance (`$near`, `$geoNear`). Un index **haché** range l'empreinte (*hash*) de la valeur, un nombre qui disperse les valeurs voisines : il ne sert qu'aux égalités, et il répartit les documents entre serveurs (partitionnement, chapitre 4 ; chapitre 6 dans la version complète). Un index **générique** (`"champ.$**"`) indexe tous les sous-champs d'un champ, même inconnus à l'avance.

Exemple guidé

1. Texte : indexez le libellé, puis cherchez un mot, puis deux :

```
db.produits.createIndex({ libelle: "text" })
db.produits.countDocuments({ $text: { $search: "sport" } })
db.produits.countDocuments({ $text: { $search: "sport livres" } })
```

Vous devez obtenir

`libelle_text`, puis `40` et `80`.

2. Géographique : rangez cinq magasins, indexez leur position, puis cherchez ceux qui sont à moins de 200 km de Rabat, et leurs distances :

```
db.magasins.insertMany([
  { _id: "Rabat",      position: { type: "Point", coordinates: [ -6.8498, 34.0209 ] } },
  { _id: "Casablanca", position: { type: "Point", coordinates: [ -7.5898, 33.5731 ] } },
  { _id: "Fès",        position: { type: "Point", coordinates: [ -5.0078, 34.0181 ] } },
  { _id: "Tanger",     position: { type: "Point", coordinates: [ -5.8340, 35.7595 ] } },
  { _id: "Marrakech",  position: { type: "Point", coordinates: [ -7.9811, 31.6295 ] } } ])
db.magasins.createIndex({ position: "2dsphere" })
db.magasins.find({ position: { $near: {
                    $geometry: { type: "Point", coordinates: [ -6.8498, 34.0209 ] },
                    $maxDistance: 200000 } } }, { _id: 1 })
db.magasins.aggregate([
  { $geoNear: { near: { type: "Point", coordinates: [ -6.8498, 34.0209 ] },
                distanceField: "km", distanceMultiplier: 0.001 } },
  { $project: { km: { $round: [ "$km", 0 ] } } } ])
```

Vous devez obtenir

```
position_2dsphere
[ { _id: 'Rabat' }, { _id: 'Casablanca' }, { _id: 'Fès' } ]
[ { _id: 'Rabat', km: 0 }, { _id: 'Casablanca', km: 85 }, { _id: 'Fès', km: 170 },
  { _id: 'Tanger', km: 215 }, { _id: 'Marrakech', km: 286 } ]
```

`$maxDistance` et les distances sont en mètres ; `distanceMultiplier: 0.001` les convertit en kilomètres.

3. Haché : indexez le client des commandes, puis cherchez une égalité et une plage :

```
db.commandes.createIndex({ "client.id": "hashed" })
resume(db.commandes.find({ "client.id": 42 }).explain("executionStats"))
resume(db.commandes.find({ "client.id": { $gt: 95 } }).explain("executionStats"))
```

4. Générique : rangez trois articles aux attributs différents, puis indexez tous leurs attributs :

```
db.essai.insertMany([
  { ref: "A", attributs: { couleur: "rouge", poids: 2 } },
  { ref: "B", attributs: { couleur: "bleu", taille: "M" } },
  { ref: "C", attributs: { matiere: "bois" } } ])
db.essai.createIndex({ "attributs.$**": 1 })
resume(db.essai.find({ "attributs.couleur": "rouge" }).explain("executionStats"))
resume(db.essai.find({ "attributs.matiere": "bois" }).explain("executionStats"))
```

À vous

a. Pourquoi la recherche « sport livres » trouve-t-elle 80 produits ?
b. Dans quel ordre `$near` renvoie-t-il les magasins ? Pourquoi Tanger n'est-il pas dans la réponse ?
c. Qu'affichent les deux recherches de l'étape 3 ? Pourquoi l'index haché ne sert-il pas à la plage ?
d. Quel index sert les deux recherches de l'étape 4 ? Quel serait l'intérêt d'un tel index pour un catalogue dont les produits ont des attributs différents ?

Exercice 25 : Choisir un index : quatre requêtes

Pour chaque requête, proposez un index, créez-le, puis vérifiez avec `resume` qu'il évite le tri en mémoire, et dites si la requête est couverte (exercice 9). Préparez d'abord la collection : les commandes reçoivent leur total, et leurs index sont supprimés.

```
db.commandes.updateMany({}, [ { $set: { total: { $sum: { $map: { input: "$lignes", as: "l",
                                  in: { $multiply: [ "$$l.prix", "$$l.qte" ] } } } } } } ])
db.commandes.dropIndexes()
```

```
// a.
db.commandes.find({ statut: "payée", "client.ville": "Fès" }).sort({ date: -1 })
// b.
db.commandes.find({ "client.id": 42 }, { _id: 0, "client.id": 1, date: 1 })
// c.
db.commandes.find({ date: { $gte: ISODate("2026-06-01") },
                    "client.ville": "Tanger" }).sort({ total: -1 })
// d.
db.produits.find({ tags: "promo", stock: { $gt: 0 } })
```

Pour finir

Sur le disque, la collection `commandes_volume` occupe une quinzaine de mégaoctets, plus ses index. Supprimez-la si vous voulez libérer la place : `db.commandes_volume.drop()`. Gardez la base `boutique` ; pour la remettre dans son état de départ, relancez `donnees-boutique.js` : il supprime toute la base, y compris `commandes_volume` et les collections créées dans la partie 4 (`stats_villes`, `rapport_villes`, `essai_sortie`, `ventes_jour`, `rapport_mensuel`, `tranches`) et dans la partie 5 (`magasins`, et `essai` ou `copie` si vous les avez gardées).

Mémo : commandes et fonctions

Dans PyMongo, une étape, un accumulateur ou un opérateur s'écrivent comme dans `mongosh`, sous la forme d'un dictionnaire Python dont les clés sont entre guillemets : `{"$match": {"statut": "payée"}}`. Les tableaux 1 à 3 valent donc pour les deux ; les tableaux 4 à 6 mettent en regard les méthodes de `mongosh` et les fonctions de PyMongo ; le tableau 6 rassemble les commandes de la partie 5. Dans les exemples, `c` désigne une collection (`db.commandes` dans `mongosh`, `db.commandes` ou `db["commandes"]` en Python).

1. Les étapes d'une chaîne d'agrégation

Étape	Ce qu'elle fait	Exemple	En SQL
`$match`	garde les documents qui vérifient le filtre ; en tête de chaîne, elle profite des index	`{ $match: { statut: "payée" } }`	`WHERE` ; `HAVING` après `$group`
`$group`	un document par valeur de `_id` (`_id: null` : un seul groupe) ; les autres champs viennent d'accumulateurs	`{ $group: { _id: "$client.ville", n: { $sum: 1 } } }`	`GROUP BY`
`$project`	garde (1) ou masque (0) des champs, en renomme, en calcule	`{ $project: { _id: 0, ville: "$client.ville", total: 1 } }`	`SELECT a, b AS c`
`$set`	ajoute ou remplace un champ et garde tous les autres	`{ $set: { nb_lignes: { $size: "$lignes" } } }`	colonne calculée
`$unset`	retire des champs	`{ $unset: [ "lignes", "client" ] }`	
`$sort`	trie (1 : croissant, -1 : décroissant) ; un second champ départage	`{ $sort: { ca: -1, _id: 1 } }`	`ORDER BY`
`$skip`, `$limit`	saute, garde les *n* premiers documents	`{ $skip: 10 }, { $limit: 5 }`	`OFFSET`, `LIMIT`
`$count`	un seul document : le nombre de documents reçus	`{ $count: "commandes" }`	`COUNT(*)`
`$sortByCount`	regroupe, compte, trie par nombre décroissant	`{ $sortByCount: "$client.ville" }`	`GROUP BY`, puis `ORDER BY COUNT(*)`
`$unwind`	une copie du document par élément du tableau	`{ $unwind: "$lignes" }`	(aucun)
`$lookup`	ajoute, dans un tableau, les documents d'une autre collection dont `foreignField` égale `localField`	`{ $lookup: { from: "produits", localField: "lignes.ref", foreignField: "ref", as: "produit" } }`	`LEFT JOIN`
`$bucket`	range les documents par tranches de valeurs (champ `count` par défaut)	`{ $bucket: { groupBy: "$total", boundaries: [ 0, 1000, 2000, 5000 ] } }`	`GROUP BY CASE …`
`$facet`	plusieurs chaînes sur les mêmes documents ; un champ de sortie par chaîne	`{ $facet: { par_statut: [ { $sortByCount: "$statut" } ], total: [ { $count: "n" } ] } }`	plusieurs requêtes
`$merge`	écrit le résultat dans une collection : remplace les documents de même `_id`, insère les autres, garde le reste	`{ $merge: { into: "stats_villes", whenMatched: "replace" } }`	`MERGE INTO`
`$out`	remplace toute une collection par le résultat	`{ $out: "tranches" }`	`CREATE TABLE … AS SELECT`
`$indexStats`	un document par index, avec son nombre d'utilisations	`{ $indexStats: {} }`	(vue système)

2. Les accumulateurs, dans $group (et dans le champ output de $bucket)

Accumulateur	Ce qu'il calcule pour chaque groupe	En SQL
`{ $sum: 1 }`	le nombre de documents	`COUNT(*)`
`{ $sum: "$total" }`	la somme d'un champ	`SUM(total)`
`{ $avg: "$total" }`	la moyenne	`AVG(total)`
`{ $min: "$date" }, { $max: "$date" }`	la plus petite, la plus grande valeur	`MIN`, `MAX`
`{ $first: "$client.nom" } ; $last`	la valeur du premier, du dernier document arrivé	
`{ $push: "$_id" }`	le tableau des valeurs, doublons compris	`ARRAY_AGG(x)`
`{ $addToSet: "$_id" }`	le tableau des valeurs distinctes	`ARRAY_AGG(DISTINCT x)`

3. Les opérateurs de filtre et de calcul

Opérateur	Rôle	Exemple
Dans un filtre (`$match`*,* `find`*,* `countDocuments`*)*		
`$gt`, `$gte`, `$lt`, `$lte`	supérieur, supérieur ou égal, inférieur, inférieur ou égal	`{ total: { $gte: 3000, $lt: 3500 } }`
`$ne`, `$in`	différent de ; égal à l'une des valeurs	`{ statut: { $ne: "payée" } }, { _id: { $in: [ 10001, 10002 ] } }`
Dans un calcul (`$project`*,* `$set`*,* `$group`*) :* `"$champ"` *est la valeur d'un champ,* `"$$l"` *celle d'une variable*		
`$multiply`	le produit de ses arguments	`{ $multiply: [ "$lignes.prix", "$lignes.qte" ] }`
`$map`	applique un calcul à chaque élément d'un tableau	`{ $map: { input: "$lignes", as: "l", in: { $multiply: [ "$$l.prix", "$$l.qte" ] } } }`
`$sum` sur un tableau	la somme de ses éléments	`{ $sum: { $map: … } }`
`$size`	le nombre d'éléments d'un tableau	`{ $size: "$lignes" }`
`$round`	arrondit à *n* décimales	`{ $round: [ "$panier", 0 ] }`
`$dateToString`	écrit une date selon un format	`{ $dateToString: { format: "%Y-%m", date: "$date" } }`

4. Index, plans d'exécution et administration

mongosh	PyMongo	Effet
`c.createIndex({ a: 1 })`	`c.create_index("a")`	index simple ; renvoie son nom, `a_1`
`c.createIndex({ a: 1, b: -1 })`	`c.create_index([("a", 1), ("b", -1)])`	index composé ; ordre des champs : règle ESR
`c.createIndex({ a: 1 }, { unique: true })`	`c.create_index("a", unique=True)`	index unique : refuse les doublons (erreur E11000)
`c.createIndex({ d: 1 }, { expireAfterSeconds: 1800 })`	`c.create_index("d", expireAfterSeconds=1800)`	index TTL : supprime les documents périmés
`c.createIndex({ e: -1 }, { partialFilterExpression: { statut: "payée" } })`	`c.create_index([("e", -1)], partialFilterExpression={"statut": "payée"})`	index partiel : seulement les documents du filtre
`c.createIndex({ f: 1, g: -1 }, { name: "F_G" })`	`c.create_index([("f", 1), ("g", -1)], name="F_G")`	un nom choisi
`c.getIndexes()`	`c.list_indexes(), c.index_information()`	la liste des index
`c.dropIndex("a_1") ; c.dropIndexes()`	`c.drop_index("a_1") ; c.drop_indexes()`	supprime un index ; tous, sauf `_id_`
`c.find(f).hint("a_1")`	`c.find(f).hint("a_1") ; count_documents(f, hint="a_1")`	impose un index
`c.find(f).hint({ $natural: 1 })`	`c.find(f).hint([("$natural", 1)])`	impose un parcours de la collection, sans index
`c.find(f).explain("executionStats")`	`c.find(f).explain()`	plan gagnant et compteurs (`nReturned`, `totalKeysExamined`, `totalDocsExamined`, `executionTimeMillis`)
`c.explain("executionStats") .aggregate(ch)`	`db.command("explain", {"aggregate": "commandes", "pipeline": ch, "cursor": {}}, verbosity="executionStats")`	plan d'une chaîne, étape par étape (champ `stages`)
`c.stats().indexSizes`	`next(c.aggregate([{"$collStats": {"storageStats": {}}}])) ["storageStats"]["indexSizes"]`	la place de chaque index, en octets
`c.validate({ full: true }) .keysPerIndex`	`db.command("validate", "produits", full=True)["keysPerIndex"]`	vérifie la collection ; le nombre de clés de chaque index
`c.aggregate([ { $indexStats: {} } ])`	`c.aggregate([{"$indexStats": {}}])`	le nombre d'utilisations de chaque index
`db.setProfilingLevel(1, { slowms: 20 })`	`db.command("profile", 1, slowms=20)`	profileur : 0 arrêté, 1 opérations plus lentes que `slowms`, 2 toutes
`db.getProfilingStatus()`	`db.command("profile", -1)`	l'état du profileur
`db.system.profile.find()`	`db["system.profile"].find()`	les traces enregistrées
`db.version()`	`client.server_info()["version"]`	la version du serveur

5. Lire et écrire depuis Python

mongosh	PyMongo	Effet, réponse
`mongosh`	`client = MongoClient( "mongodb://localhost:27017")`	crée le client sans attendre ; un serveur absent n'est signalé qu'à la première commande (au bout de 30 s par défaut). Depuis le conteneur `python-mongo` : `mongodb://mongo:27017`
`use boutique`	`db = client["boutique"]`	choisit la base
`show collections`	`db.list_collection_names()`	les noms des collections
`c.find(f, p).sort({ a: -1, b: 1 }) .skip(n).limit(n)`	`c.find(f, p).sort([("a", -1), ("b", 1)]) .skip(n).limit(n)`	un curseur, parcouru par `for` ; un seul champ : `sort("a", -1)`
`c.findOne(f)`	`c.find_one(f)`	un dictionnaire, ou `None`
`c.countDocuments(f) ; c.distinct("x")`	`c.count_documents(f) ; c.distinct("x")`	un nombre ; la liste des valeurs distinctes
`c.aggregate(ch)`	`c.aggregate(ch)`	`ch` : liste de dictionnaires ; un curseur
`c.insertOne(d) ; c.insertMany(l)`	`c.insert_one(d) ; c.insert_many(l)`	`inserted_id` ; `inserted_ids`
`c.updateOne(f, m) ; c.updateMany(f, m)`	`c.update_one(f, m) ; c.update_many(f, m) ; upsert=True`	`matched_count`, `modified_count`, `upserted_id`
`c.deleteOne(f) ; c.deleteMany(f)`	`c.delete_one(f) ; c.delete_many(f)`	`deleted_count` ; les index restent
`c.bulkWrite([ { insertOne: { document: d } }, { updateOne: { filter: f, update: m } } ])`	`c.bulk_write([InsertOne(d), UpdateOne(f, m)], ordered=True)`	`inserted_count`, `modified_count`, `upserted_count` ; un envoi par série d'opérations de même nature
`c.drop()`	`c.drop()`	supprime la collection et ses index
`Decimal128("3.50")`	`Decimal128("3.50") ; x.to_decimal()`	`from bson import Decimal128` ; `to_decimal()` donne un `Decimal`
`ISODate("2026-07-01")`	`datetime(2026, 7, 1)`	`from datetime import datetime` ; heure UTC
`E11000 duplicate key error`	`DuplicateKeyError ; BulkWriteError`	module `pymongo.errors` ; dans un lot : `details["writeErrors"]`
`(serveur injoignable)`	`ConnectionFailure, ServerSelectionTimeoutError`	module `pymongo.errors`

6. Index : rangement, coût, plans et index spécialisés (partie 5)

mongosh	PyMongo	Effet
`c.find(f, p).showRecordId()`	`c.find(f, p, show_record_id=True)`	ajoute le RecordId de chaque document
`c.stats({ indexDetails: true }) .indexDetails["a_1"].uri`	–	le fichier `.wt` d'un index
`db.adminCommand({ fsync: 1 })`	`client.admin.command("fsync")`	écrit tout de suite les fichiers sur le disque
`c.totalIndexSize()`	`next(c.aggregate([{"$collStats": {"storageStats": {}}}])) ["storageStats"]["totalIndexSize"]`	la place de tous les index, en octets
`db.system.profile.find({ op: "insert" }, { keysInserted: 1 })`	`db["system.profile"].find({"op": "insert"}, {"keysInserted": 1})`	clés d'index écrites par chaque insertion (profileur au niveau 2)
`Date.now()`	`time.perf_counter()` (en secondes ; `import time`)	l'heure, pour mesurer une durée
`c.find(f).explain("allPlansExecution")`	`db.command("explain", {"find": "commandes", "filter": f}, verbosity= "allPlansExecution")`	le plan, avec les plans écartés (`rejectedPlans`)
`{ libelle: /^Livres/ } ; /n°12/ ; /^livres/i`	`{"libelle": {"$regex": "^Livres"}} ; {"$regex": "^livres", "$options": "i"}`	ancrée au début : une plage de clés ; sinon, toutes les clés
`c.createIndex({ libelle: "text" })`	`c.create_index([("libelle", "text")])`	index texte (un seul par collection)
`c.find({ $text: { $search: "sport livres" } })`	`c.find({"$text": {"$search": "sport livres"}})`	les documents qui contiennent l'un des mots
`c.createIndex({ position: "2dsphere" })`	`c.create_index([("position", "2dsphere")])`	index géographique ; position en GeoJSON, `[ longitude, latitude ]`
`{ position: { $near: { $geometry: pt, $maxDistance: 200000 } } }`	le même filtre, en dictionnaire	du plus proche au plus lointain, à moins de 200 km
`{ $geoNear: { near: pt, distanceField: "km", distanceMultiplier: 0.001 } }`	la même étape	première étape d'une chaîne : ajoute la distance
`c.createIndex({ "client.id": "hashed" })`	`c.create_index([("client.id", "hashed")])`	index haché : égalités seulement
`c.createIndex({ "attributs.$**": 1 })`	`c.create_index([("attributs.$**", 1)])`	index générique : tous les sous-champs

Erreurs fréquentes

Les messages tels qu'ils s'affichent, leur cause et ce qu'il faut faire.

Message ou symptôme	Cause	Que faire
`docker: Error response from daemon: Conflict. The container name "/mongo" is already in use by container "5105…"`	un conteneur de ce nom existe déjà, même arrêté	`docker start mongo` pour le reprendre ; ou `docker rm -f mongo`, puis le recréer
`… Bind for 0.0.0.0:27017 failed: port is already allocated`	le port de votre machine est déjà pris (un MongoDB installé, un autre conteneur) ; le conteneur est créé, mais pas démarré	arrêter l'autre serveur, ou publier un autre port (`-p 27018:27017`) ; puis `docker rm` du conteneur créé
`Error response from daemon: No such container: mongoo`	faute de frappe dans le nom, ou conteneur supprimé	`docker ps -a` affiche les noms
`Error response from daemon: container 5105… is not running`	conteneur arrêté (redémarrage de la machine, `docker stop`)	`docker start mongo`
`failed to connect to the docker API at unix://… ; check if the path is correct and if the daemon is running`	Docker Desktop n'est pas lancé	lancer Docker Desktop, attendre qu'il soit prêt, recommencer
`MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017`	aucun serveur n'écoute sur ce port	démarrer le serveur ; vérifier le port
`MongoServerError[Location40234]: The field 'total' must be an accumulator object`	dans `$group`, un champ sans accumulateur : `total: "$prix"`	`total: { $sum: "$prix" }` (exercice 2)
`[ { _id: null, total: 0 } ]`, sans erreur	`$sum: "prix"` sans `$` : un texte constant, qui compte pour 0 ; et `_id: null`, un champ de regroupement absent ou mal écrit	`"$prix"`, `"$categorie"` : le `$` désigne la valeur du champ
`SyntaxError: Unexpected token, expected "," (1:32)`	crochet, accolade ou virgule mal placée dans la chaîne d'étapes	une étape par ligne, chaque étape entre accolades, séparées par des virgules
`Invalid REPL keyword`	ligne qui commence par `.sort(` ou `.explain(` après une instruction complète	écrire la chaîne sur une ligne, ou couper à l'intérieur d'une parenthèse ouverte
`MongoServerError[BadValue]: unknown operator: $gtt`	opérateur mal écrit	`$gt`, `$gte`, `$in`…
`MongoServerError[IndexKeySpecsConflict]: An existing index has the same name as the requested index. …`	un index sur la même clé existe déjà, avec d'autres options (`unique`)	`db.c.getIndexes()` ; supprimer l'ancien (`dropIndex`) ou donner un autre nom (exercice 17)
`MongoServerError[BadValue]: error processing query: … hint provided does not correspond to an existing index`	`hint()` avec un nom d'index qui n'existe pas	`db.c.getIndexes()` donne les noms (`ref_1`…)
`0`, sans erreur, après `db.Commandes.countDocuments()`	les noms distinguent les majuscules	`show collections` ; recopier le nom exact
`pymongo.errors.ServerSelectionTimeoutError: localhost:27017: [Errno 111] Connection refused …`	avec Docker, `localhost` désigne le conteneur Python lui-même	dans `python-mongo` : `mongodb://mongo:27017`
`bson.errors.InvalidDocument: Invalid document: cannot encode object: Decimal('47.00') …`	un `Decimal` de Python dans un filtre ou un document	`Decimal128("47.00")`

Messages obtenus le 21 et le 22 octobre 2026 avec MongoDB 8.0.32, mongosh 2.12.0 et PyMongo 4.19.0 ; seuls les identifiants de conteneur, les noms et les ports changent.